



Escuela Técnica Superior de
Ingeniería Informática

Generación y Adquisición de Señal Senoidal en STM32G474RE

Grado en Ingeniería Informática
Universidad de Sevilla

Autor: Rafael García Ruiz
Director: Pablo Pérez García

2025/2026

Resumen

El presente Trabajo de Fin de Grado aborda el diseño y desarrollo de un sistema embebido para la generación y adquisición de señales senoidales aplicado al estudio de impedancias. El sistema se implementa sobre el microcontrolador **STM32G474RE**, aprovechando sus periféricos internos con el objetivo de reducir la necesidad de componentes externos y obtener una solución compacta, eficiente y de bajo coste.

El funcionamiento del dispositivo se basa en la generación de una señal senoidal mediante el convertidor digital-analógico (**DAC**) del microcontrolador. Dicha señal se obtiene a partir de una tabla de muestras precalculada y se transmite al periférico mediante **DMA**, utilizando un temporizador interno como fuente de disparo para garantizar una frecuencia de actualización estable. La señal generada se aplica posteriormente a una muestra o carga de prueba, cuya respuesta es adquirida mediante el convertidor analógico-digital (**ADC**), también asistido por **DMA**.

Una parte fundamental del proyecto consiste en la sincronización entre la generación y la adquisición de la señal. Para ello se emplea el temporizador **TIM6**, que permite coordinar el funcionamiento del DAC y del ADC mediante eventos hardware, reduciendo la intervención de la CPU y minimizando posibles errores derivados de la latencia del software.

El firmware del sistema se desarrolla en **Rust** en un entorno *bare-metal*, sin sistema operativo ni biblioteca estándar. Esta elección permite explorar el uso de un lenguaje moderno en sistemas embebidos, destacando sus ventajas en cuanto a seguridad de memoria, control de recursos y robustez del código. Durante el desarrollo se configura una infraestructura completa de compilación cruzada, carga y depuración mediante herramientas como **Cargo**, **probe-rs** y **defmt**.

Finalmente, el sistema se valida mediante pruebas experimentales orientadas a comprobar la correcta generación de la señal senoidal, la adquisición de muestras mediante el ADC y la estabilidad del funcionamiento basado en DMA. El proyecto demuestra la viabilidad de utilizar el STM32G474RE con Rust como plataforma integrada para aplicaciones de generación, adquisición y análisis de señales analógicas, sentando las bases para futuros desarrollos relacionados con la caracterización eléctrica de materiales o tejidos biológicos.

Palabras clave: STM32G474RE, Rust, bare-metal, no-std, bioimpedancia, DAC, ADC, DMA, TIM6, sistemas embebidos, señal senoidal.

Índice

Resumen	2
Glosario de términos y acrónimos	4
1. Introducción del proyecto	5
1.1. Resumen de objetivos	5
2. Descripción del proyecto	5
2.1. Estado del arte	6
2.1.1. Fundamentos teóricos	6
2.1.2. Medición de la impedancia	7
2.2. Objetivos del proyecto	9
3. Planificación del proyecto	10
3.1. Análisis de requisitos	10
3.2. Elección de componentes y herramientas	11
3.2.1. Microcontrolador	11
3.2.2. Resistencias	12
3.2.3. Software	12
3.2.4. Infraestructura de compilación y ejecución	14
3.2.5. Ejecución del firmware y organización de memoria	16
3.2.6. Instrumentación de laboratorio	18
3.2.7. Otros componentes considerados	18
3.3. Análisis de costes estimados	18
3.4. Planificación de hitos	19
3.4.1. H1 – Aprendizaje y entorno Rust embebido funcional	19
3.4.2. H2 – Generación de señal senoidal mediante DAC	20
3.4.3. H3 – Adquisición de señal mediante ADC	20
3.4.4. H4 – Validación experimental con Analog Discovery 3	20
3.4.5. H5 – Documentación y cierre del proyecto	21
3.4.6. Resumen temporal	21
3.4.7. Diagrama de Gantt	22
4. Desarrollo del proyecto	23
4.1. Fundamentos del Sistema Microcontrolador	23
4.2. Diseño del sistema	23
4.2.1. Funcionamiento integrado	23
4.3. Desarrollo del sistema (Fases)	23
4.3.1. F1: Planteamiento y estudio inicial	24
4.3.2. F2: Aprendizaje de Rust embebido e infraestructura	24
4.3.3. F3: Estudio del STM32G474RE y periféricos analógicos	24
4.3.4. F4: Prototipos mínimos de GPIO, DAC y ADC	24
4.3.5. F5: Integración DAC-DMA-TIM6	24
4.3.6. F6: Integración ADC-DMA y lectura de muestras	25
4.3.7. F7: Selección de perfiles de frecuencia mediante pulsador	25
4.3.8. F8: Validación experimental con carga resistiva-capacitiva	26
4.4. Seguimiento de la planificación	31
4.5. Plan de pruebas	31

5. Cierre del documento	31
5.1. Conclusiones	31
5.2. Galería de figuras	32
Bibliografía	33
A. Anexos	34
A.1. Implementación del Firmware en Rust	34
A.2. Configuración de Periféricos y Bucle Principal	34
A.3. Gestión de perfiles de frecuencia y pulsador de usuario	41
A.4. Detalles de la Síntesis Digital Directa (DDS)	41
A.5. Uso de inteligencia artificial generativa	41

Glosario de términos y acrónimos

ADC (Analog-to-Digital Converter): Periférico que convierte una señal analógica de tensión en un valor digital numérico.

DAC (Digital-to-Analog Converter): Periférico que convierte un valor numérico digital en una señal de tensión analógica.

DMA (Direct Memory Access): Mecanismo que permite a los periféricos acceder a la memoria RAM independientemente de la CPU, reduciendo la carga de procesamiento.

HAL (Hardware Abstraction Layer): Capa de software que permite interactuar con el hardware del microcontrolador sin necesidad de manipular registros directamente.

Impedancia (Z): Medida de oposición total que presenta un circuito al paso de una corriente alterna.

Jitter: Variabilidad temporal en la ejecución de un evento o en la llegada de una señal, crítica en sistemas de tiempo real.

no_std: Atributo de Rust que indica que el programa no utiliza la biblioteca estándar (std), permitiendo su ejecución en sistemas sin sistema operativo (bare-metal).

Ownership: Concepto central de Rust que gestiona la seguridad de memoria mediante reglas de propiedad y préstamo de variables.

PCB (Printed Circuit Board): Placa de circuito impreso donde se montan y conectan los componentes electrónicos.

RCC (Reset and Clock Control): Módulo encargado de gestionar los relojes y el reinicio de los periféricos en el STM32.

UART (Universal Asynchronous Receiver-Transmitter): Protocolo de comunicación serie utilizado habitualmente para transmitir datos a un ordenador.

1. Introducción del proyecto

El análisis de las propiedades eléctricas de los tejidos biológicos, conocido como bioimpedancia, se ha consolidado como una herramienta fundamental en la medicina moderna para la caracterización de estados fisiológicos y patológicos. Sin embargo, el diseño de instrumentación precisa y de bajo coste para este propósito presenta desafíos significativos en la sincronización de señales y en la fiabilidad del software embebido.

El presente Trabajo de Fin de Grado aborda el diseño, desarrollo e implementación de un sistema de adquisición de señales biomédicas basado en el microcontrolador de alto rendimiento **STM32G474RE**. La propuesta se diferencia de las soluciones tradicionales por el uso del lenguaje de programación **Rust**, cuya seguridad de memoria y gestión de recursos sin tiempo de ejecución (*no-std*) ofrece una robustez superior para aplicaciones críticas.



Figura 1: Diagrama conceptual del sistema

1.1. Resumen de objetivos

El objetivo del proyecto es desarrollar una plataforma embebida capaz de generar una señal senoidal mediante el DAC del STM32G474RE y adquirir su respuesta mediante el ADC, utilizando Rust como lenguaje de implementación y periféricos internos del microcontrolador para reducir la intervención de la CPU.

- **Desarrollo de un motor de síntesis digital:** Implementar la generación de ondas senoidales mediante el uso coordinado de tablas en memoria, acceso directo a memoria (DMA) y convertidores digital-analógico (DAC).
- **Sincronización de hardware:** Configurar un sistema de disparo (*triggering*) basado en temporizadores para garantizar que la adquisición (ADC) y la generación (DAC) ocurran en fase, eliminando el error por latencia de software.
- **Implementación en Rust embebido:** Explorar y validar el uso de Rust en un entorno *bare-metal*, accediendo a los registros del microcontrolador mediante la capa PAC disponible para la familia STM32G4.
- **Monitorización de muestras:** Establecer un flujo de datos eficiente para observar las muestras adquiridas en tiempo real mediante `defmt`, RTT y `probe-rs`.

A través de estos objetivos, el proyecto busca demostrar que es posible desarrollar un prototipo funcional de instrumentación embebida utilizando componentes de bajo coste y lenguajes de programación modernos.

2. Descripción del proyecto

En este proyecto se aborda el diseño y desarrollo de un sistema de adquisición biomédica basado en un microcontrolador **STM32G474RE**. Su función principal es generar una señal senoidal controlada mediante el DAC y adquirir la señal resultante mediante el ADC tras atravesar una muestra o carga de prueba. La comparación externa de ambas señales permite observar efectos relacionados con sus propiedades eléctricas, tales como atenuación, filtrado o variaciones de amplitud asociadas a la impedancia.

La coordinación entre generación y adquisición se realiza mediante **TIM6** y **DMA**. El temporizador proporciona el evento periódico de disparo, mientras que el DMA transfiere muestras hacia el DAC y desde el ADC sin requerir una intervención continua de la CPU.

El diseño del sistema se inspira en enfoques previamente utilizados en instrumentación biomédica basada en microcontroladores y análisis de señal aplicada al estudio de propiedades tisulares.

El dispositivo se divide en dos bloques funcionales principales:

- **Generación de la señal:** producción de una onda senoidal a partir de una tabla precalculada de 12 bits, transferida al DAC mediante DMA y temporizada por TIM6.
- **Lectura y monitorización:** adquisición sincronizada mediante ADC y almacenamiento de muestras en memoria mediante DMA, con visualización posterior mediante trazas de depuración.

Durante el desarrollo del proyecto se busca implementar la mayor parte del sistema utilizando los periféricos internos del microcontrolador, reduciendo la necesidad de componentes externos.

2.1. Estado del arte

2.1.1. Fundamentos teóricos

Concepto de impedancia

La **impedancia** (Z) es la medida de la oposición que presenta un circuito al paso de una corriente cuando se aplica una tensión alterna [1]. Constituye un parámetro fundamental en el análisis de sistemas eléctricos y electrónicos, ya que describe la relación entre el voltaje y la corriente en régimen sinusoidal. Su definición básica es:

$$Z = \frac{V}{I} \quad [\Omega]$$

A diferencia de la corriente continua (CC), donde únicamente interviene la resistencia, en corriente alterna (CA) la respuesta del circuito depende también de la **frecuencia** de la señal. Esto se debe a la presencia de elementos *reactivos* (condensadores e inductancias), que modifican tanto la amplitud como la fase de la corriente respecto al voltaje aplicado.

Por ello, la impedancia combina dos componentes y se expresa como un número complejo:

$$Z = R + jX \quad [\Omega]$$

donde:

- R es la parte real, correspondiente a la **resistencia**, que describe la disipación de energía.
- X es la parte imaginaria, la **reactancia**, asociada a los efectos capacitivos e inductivos que provocan variaciones en la fase de la corriente.

En consecuencia, la impedancia se representa de forma vectorial incorporando tanto magnitud como fase, características esenciales para el análisis de señales sinusoidales.

Concepto de fasor

Las señales presentes en los circuitos de corriente alterna suelen ser de naturaleza sinusoidal. Para simplificar su análisis, se emplea la representación mediante **fasores** [2], que transforma magnitudes dependientes del tiempo en números complejos de módulo y fase constante.

Una tensión sinusoidal puede expresarse como:

$$v(t) = V_{\max} \cdot \sin(\omega t)$$

donde:

- $v(t)$: tensión instantánea.
- $V_{\max}$: amplitud o valor máximo de la señal.
- ωt : argumento o posición angular de la onda, siendo ω la frecuencia angular.

Los fasores permiten representar esta señal mediante un número complejo cuya:

- **magnitud** se corresponde con la amplitud (o valor eficaz) de la onda,
- **fase** describe el desplazamiento angular respecto a una referencia.

Dado que la impedancia también se expresa como un número complejo, la representación fasorial simplifica notablemente el análisis de circuitos en régimen permanente sinusoidal. Por ello constituye una herramienta fundamental para describir impedancias y estudiar el comportamiento eléctrico de materiales, tejidos o componentes electrónicos.

2.1.2. Medición de la impedancia

La medición de impedancia es fundamental para analizar y diseñar circuitos eléctricos y electrónicos. Su valor proporciona información clave sobre el comportamiento del sistema ante diferentes frecuencias, especialmente debido a que la impedancia depende de la reactancia, la cual varía con la frecuencia. Por ello, resulta necesario realizar mediciones en un rango adecuado de frecuencias para identificar posibles fallos, optimizar el rendimiento del circuito y evaluar su comportamiento dinámico.

Dado que la impedancia es un valor complejo, su medición requiere instrumentos o técnicas específicas que permitan determinar tanto su magnitud como su fase.

Generador de señales y osciloscopio

Una técnica básica de medición consiste en generar una señal conocida, generalmente sinusoidal, mediante un generador de señales. La señal se aplica al componente bajo prueba, observando la respuesta mediante un osciloscopio. Este método permite comparar la amplitud, fase y forma de la señal aplicada con la señal resultante, proporcionando una evaluación cualitativa del comportamiento del circuito.

Medición LCR o puente de impedancia

El puente LCR permite medir impedancias con gran precisión. El método compara la impedancia desconocida con elementos de impedancia conocida. Ajustando estos elementos hasta equilibrar el puente, es posible determinar la impedancia del componente bajo prueba con elevada exactitud.

Medición en dos puntos

Este método mide la resistencia o impedancia del elemento bajo prueba en serie con una resistencia auxiliar. Se asume que la resistencia auxiliar es despreciable frente a la del elemento medido. Aunque es un método sencillo, puede producir errores si ambas resistencias son comparables. Debido a su simplicidad, este será el método utilizado en nuestro proyecto.

Medición en tres puntos

En este caso se utilizan dos resistencias auxiliares adicionales. Se realizan tres mediciones: R_{12} , R_{23} y R_{13} . La resistencia buscada se calcula mediante:

$$R_1 = \frac{R_{12} - R_{23} + R_{13}}{2}$$

Este método mejora la precisión respecto al de dos puntos, aunque puede presentar errores si las resistencias auxiliares son demasiado altas o si no se mantiene una separación adecuada entre puntos de medición.

Medición en cuatro puntos

Este método emplea cuatro electrodos: dos para inyectar corriente y dos para medir la caída de tensión. La separación entre los electrodos de corriente y de medida reduce la influencia de las resistencias de contacto, permitiendo una medición precisa. La impedancia se calcula mediante la ley de Ohm:

$$Z = \frac{V}{I}$$

MFIA Impedance Analyzer

El MFIA es un analizador de impedancias digital de alta precisión que opera entre 1 mHz y 500 kHz, con una exactitud básica del 0.05 %. Su rango de medición abarca desde 1 m Ω hasta 1 T Ω . Sus aplicaciones incluyen ingeniería eléctrica, caracterización de materiales y bioimpedancia.

Impedimed SFB7

El dispositivo SFB7 realiza espectroscopia de bioimpedancia tetrapolar a 256 frecuencias entre 4 kHz y 1000 kHz. Emplea modelos como el de Cole y la teoría de mezcla de Hanai para estimar parámetros corporales como TBW, ECF, ICF, FFM y FM, lo que lo convierte en un estándar en estudios biomédicos.

Keysight E4980A Precision LCR Meter

El medidor LCR Keysight E4980A es ampliamente utilizado por su capacidad para medir inductancia, capacitancia y resistencia con alta precisión. Opera entre 20 Hz y 2 MHz, con una exactitud básica del 0.05 %, ofreciendo gran estabilidad y repetibilidad en un amplio rango de mediciones.

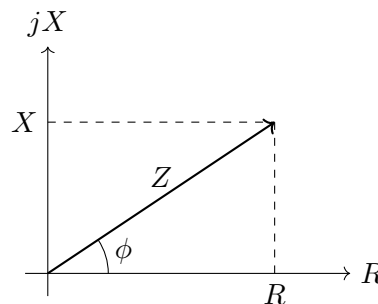


Figura 2: Representación fasorial de la impedancia compleja

2.2. Objetivos del proyecto

El objetivo principal de este proyecto es desarrollar un sistema embebido capaz de generar una señal senoidal, aplicarla a una muestra y medir la señal resultante tras atravesar dicho medio, con el fin de obtener información relacionada con sus propiedades eléctricas. Todo el sistema se implementará sobre el microcontrolador **STM32G474RE**, aprovechando al máximo sus periféricos internos para minimizar la necesidad de componentes externos.

El dispositivo deberá cumplir las siguientes características fundamentales:

- **Implementación interna:** toda la lógica del sistema y el mayor número posible de funciones deben realizarse mediante los periféricos integrados en el microcontrolador (DAC, ADC, temporizadores, DMA, etc.).
- **Generación y adquisición de señal:** el sistema debe generar una señal senoidal estable y adquirir la señal de salida tras atravesar la muestra o carga de prueba.
- **Configurabilidad:** la frecuencia de la señal generada debe poder ajustarse mediante perfiles predefinidos para permitir ensayos en distintos rangos de operación. La amplitud se mantiene fija en esta versión del prototipo mediante la tabla senoidal empleada por el DAC.

Además del objetivo principal, el desarrollo del proyecto permitirá adquirir conocimientos y habilidades en diversas áreas:

- **Microcontroladores:** estudio y configuración del STM32G474RE, programación en Rust y manejo de periféricos como DAC, ADC, temporizadores, GPIO y sistemas de reloj.
- **DMA (Direct Memory Access):** configuración del DMA para automatizar la transferencia de datos en la generación y adquisición de señal, reduciendo la carga de la CPU y aumentando el rendimiento.
- **Señal analógica:** acondicionamiento externo básico y manejo de señales alternas en el contexto de generación y adquisición con DAC y ADC.
- **Impedancia:** adquisición y análisis de datos para estudiar la impedancia de muestras, interpretación de los resultados y relación con su comportamiento eléctrico.
- **Procesamiento de ondas:** trabajo con señales sinusoidales, tablas de onda, ajuste de frecuencia y análisis externo de las señales adquiridas.
- **Montaje y soldadura:** desarrollo de un prototipo inicial sobre placa perforada, instalación y conexión de componentes externos y verificación del circuito.
- **Conversión CA/CC:** generación de una señal alterna mediante el DAC a partir de una referencia continua, y conversión de la señal resultante nuevamente a valores digitales mediante el ADC.

El alcance del proyecto incluye el estudio del estado del arte, el diseño conceptual y electrónico del sistema, la implementación y verificación de un prototipo funcional de laboratorio y la obtención de medidas experimentales con cargas conocidas. El diseño de una versión compacta o de una PCB dedicada queda planteado como una posible línea de continuación.

3. Planificación del proyecto

3.1. Análisis de requisitos

R1 – Microcontrolador como núcleo del sistema

El diseño del sistema debe implementarse empleando el microcontrolador **STM32G474RE** como elemento principal, utilizando tantos periféricos internos como sea posible con el fin de minimizar la necesidad de componentes externos.

R1.1 – DAC

El microcontrolador debe disponer de al menos un DAC funcional para generar la señal senoidal inicial que será aplicada a la muestra (requisito R2).

R1.2 – ADC

El sistema debe contar con un ADC de adecuada resolución y frecuencia de muestreo para medir la señal resultante tras atravesar la muestra. En esta versión del proyecto no es necesaria la presencia de múltiples ADC.

R1.3 – Temporizadores

El sistema debe contar con al menos un temporizador capaz de controlar tanto la generación de la señal como el disparo del ADC para asegurar una captura sincronizada. En este proyecto se emplea un único temporizador (**TIM6**) como generador de eventos TRGO.

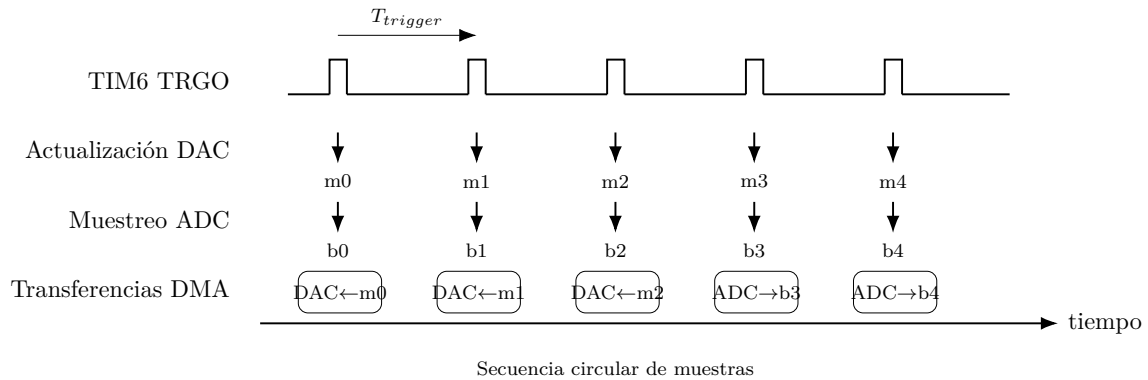


Figura 3: Sincronización entre TIM6, DAC, ADC y DMA. Cada evento de actualización de TIM6 genera el disparo que actualiza el DAC y lanza una conversión del ADC.

R2 – Generación de señal senoidal

El sistema debe generar una señal senoidal estable mediante el DAC, utilizando una tabla precalculada y transferencia mediante DMA.

R3 – Modificación de la señal

La frecuencia de la señal senoidal debe ser configurable mediante perfiles predefinidos, permitiendo realizar mediciones bajo diferentes condiciones experimentales. La amplitud queda fijada por la tabla de muestras utilizada por el DAC en esta versión del firmware.

R4 – Uso de DMA

El sistema debe emplear *Direct Memory Access* en los procesos de generación (DAC) y adquisición (ADC) con el fin de optimizar el rendimiento y minimizar la intervención del procesador.

R5 – Medición de la señal en la muestra

El sistema debe ser capaz de medir la señal tras atravesar la muestra, permitiendo analizar la atenuación y las modificaciones introducidas por ella. Para esta versión del proyecto se emplea un método de medición en dos puntos, midiendo únicamente la señal de salida.

R6 – Interpretación y transmisión de resultados

El sistema debe adquirir las muestras obtenidas mediante el ADC y transmitir las a un ordenador mediante `defmt`, `RTT` y `probe-rs`, permitiendo su visualización y análisis externo.

3.2. Elección de componentes y herramientas

3.2.1. Microcontrolador

Para cumplir los requisitos R1 y R4, el microcontrolador seleccionado debe disponer de un conjunto amplio de periféricos internos y permitir el uso de *Direct Memory Access* (DMA). Además, es necesario que dichos periféricos puedan configurarse y funcionar de manera conjunta sin generar conflictos, algo que no siempre ocurre en todos los modelos disponibles en el mercado.

El microcontrolador elegido para este proyecto es el **STM32G474RE**, basado en un núcleo **ARM Cortex-M4**. Este dispositivo resulta especialmente adecuado porque integra prácticamente todos los componentes necesarios para el proyecto (DAC, ADC, temporizadores y DMA), permitiendo que la mayor parte del diseño se implemente de forma interna sin recurrir a hardware externo adicional. Las únicas excepciones son elementos pasivos, como resistencias, que evidentemente no pueden integrarse en el microcontrolador. Para la configuración detallada de sus registros y capacidades analógicas se ha consultado el manual de referencia oficial [3].

El STM32G474RE también incorpora un sistema DMA avanzado que facilita la transferencia automática de datos entre memoria y periféricos, reduciendo significativamente la carga de procesamiento y mejorando la eficiencia general del sistema. Asimismo, la elección de un dispositivo de *STMicroelectronics* resulta ventajosa, ya que su ecosistema está ampliamente documentado y es habitual en el ámbito académico y profesional.

A continuación, se presentan los principales componentes integrados relevantes para este proyecto:

Componentes principales del STM32G474RE

- **ADCs:** 5 convertidores analógico–digital con capacidad de alta velocidad y varios canales.
- **DACs:** 7 convertidores digital–analógico capaces de generar señales precisas, incluyendo salidas de 12 bits.
- **Timers:** 17 temporizadores, entre los que se incluyen:
 - 2 temporizadores de 32 bits y 2 temporizadores de 16 bits con entradas IC/OC/PWM, contador de pulsos y modo codificador.
 - 3 temporizadores avanzados para control de motor, con hasta 8 canales PWM y generación de tiempo muerto.
 - 1 temporizador de 16 bits con 2 canales IC/OC y salida PWM auxiliar.
 - 2 temporizadores de 16 bits con funciones IC/OC/OCN/PWM y parada de emergencia.
 - 2 temporizadores de vigilancia.
 - 1 temporizador SysTick de 24 bits.
 - 2 temporizadores básicos de 16 bits.

- 1 temporizador de bajo consumo.
- **DMA:** 16 canales de *Direct Memory Access* con soporte para múltiples flujos independientes.

Otros microcontroladores fueron considerados durante la fase de selección, tales como el **STM32H743** y el **STM32L486**. Sin embargo, la disponibilidad inmediata del STM32G474RE en el departamento, hizo que estos candidatos fueran descartados en favor del modelo finalmente utilizado.

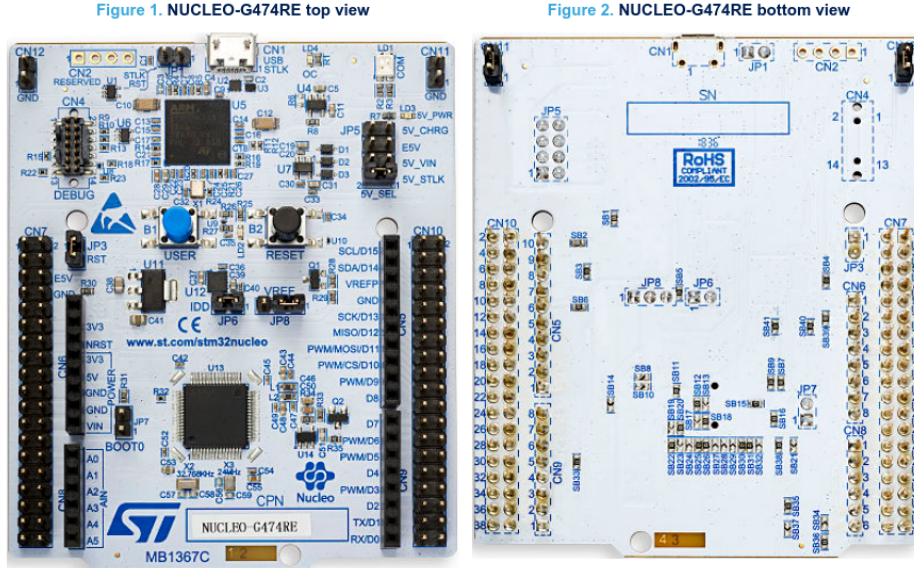


Figura 4: Microcontrolador STM32G474RE empleado en el desarrollo del sistema

3.2.2. Resistencias

Durante el desarrollo del prototipo fue necesario incorporar una serie de resistencias externas, ya que este tipo de componentes pasivos no pueden implementarse mediante los periféricos internos del microcontrolador. Para ello se emplearon resistencias SMD debido a su disponibilidad en el laboratorio, su reducido tamaño y su facilidad de integración en montajes experimentales.

Estas resistencias forman parte del acondicionamiento analógico externo del sistema y permiten adaptar la etapa de señal utilizada durante las pruebas experimentales.

3.2.3. Software

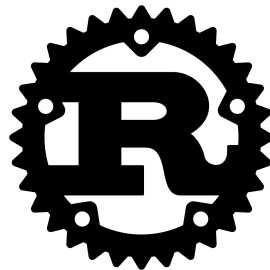


Figura 5: Logotipo del lenguaje Rust.

El desarrollo del proyecto se ha llevado a cabo utilizando herramientas orientadas al software embebido moderno, destacando el uso del lenguaje **Rust** para la programación del microcontrolador STM32G474RE. En lugar de utilizar *STM32CubeIDE*, empleado habitualmente en los

proyectos académicos sobre microcontroladores STM32, se optó por un entorno más flexible basado en editores de texto y herramientas específicas de Rust.

Los entornos de desarrollo utilizados fueron:

- **Rust**: lenguaje principal del firmware.
- **Cargo**: compilación y gestión de dependencias.
- **probe-rs**: carga y depuración sobre la placa.
- **defmt**: trazas compactas para sistemas embebidos.
- **Texmaker y LaTeX**: redacción de la memoria.
- **WaveForms**: validación experimental con Analog Discovery 3.

Ventajas del uso de Rust en sistemas embebidos

El empleo de Rust en el desarrollo del firmware ofrece una serie de beneficios significativos, destacando la **seguridad de memoria garantizada** en tiempo de compilación y su modelo de propiedad [4]. Además, para la implementación específica en sistemas embebidos, se han seguido las prácticas de gestión de periféricos documentadas en la literatura especializada [5].

- Seguridad de memoria en un entorno sin sistema operativo.
- Control de bajo nivel mediante acceso a registros sin renunciar a comprobaciones del compilador.
- Ecosistema de depuración moderno mediante **probe-rs** y **defmt**.

Desventajas y desafíos del uso de Rust

A pesar de sus ventajas, el uso de Rust en entornos embebidos presenta algunas limitaciones:

- **Ecosistema menos maduro que C/C++**, especialmente en cuanto a HALs específicos para cada microcontrolador.
- **Curva de aprendizaje más pronunciada** debido al sistema de propiedad y préstamo del lenguaje.
- **Documentación menos extensa** en comparación con marcos tradicionales como STM32Cube.
- Necesidad de una **configuración inicial más manual**, incluyendo **linker scripts**, soporte **no_std**, configuración del arranque y selección del runtime.
- Limitaciones en el uso de librerías externas escritas en C, que requieren interfaces **unsafe**.

En conjunto, Rust representa una alternativa moderna, segura y de alto rendimiento para el desarrollo embebido, y su empleo en este proyecto contribuye a explorar prácticas actuales y futuras tendencias en la industria.

3.2.4. Infraestructura de compilación y ejecución

Aunque el objetivo principal del proyecto es la generación y adquisición de señales analógicas, una parte esencial del desarrollo consistió en preparar una infraestructura de ejecución *bare-metal* en Rust. A diferencia de una aplicación convencional ejecutada sobre un sistema operativo, el firmware desarrollado se ejecuta directamente sobre el microcontrolador, sin biblioteca estándar, sin sistema de ficheros y sin servicios proporcionados por un sistema operativo.

Esto implica que el proyecto debe definir explícitamente cómo se compila el código para una arquitectura distinta a la del ordenador de desarrollo, cómo se enlaza el binario resultante en la memoria del microcontrolador, cómo se carga el programa en la placa y cómo se obtiene información de depuración durante la ejecución.

Configuración del paquete con `Cargo.toml`

El archivo `Cargo.toml` actúa como descriptor principal del proyecto Rust. En él se definen el nombre del paquete, la edición del lenguaje utilizada, los binarios disponibles, las dependencias externas y los perfiles de compilación.

En este proyecto se emplean dependencias específicas para sistemas embebidos, entre las que destacan:

- `cortex-m`: proporciona acceso a funcionalidades comunes del núcleo ARM Cortex-M.
- `cortex-m-rt`: incluye el entorno mínimo de arranque necesario para ejecutar código Rust en microcontroladores Cortex-M.
- `stm32g4`: proporciona acceso a los registros específicos de la familia STM32G4 mediante una capa PAC (*Peripheral Access Crate*).
- `defmt` y `defmt-rtt`: permiten generar trazas de depuración compactas y eficientes.
- `panic-probe`: define el comportamiento del sistema ante errores críticos o situaciones de pánico.

Durante el desarrollo se trabajó con pruebas independientes para validar progresivamente partes concretas del sistema, como la lectura del ADC, la generación mediante DAC o el funcionamiento de pines GPIO, antes de integrarlas en el firmware principal. En la versión final documentada, estas funcionalidades quedan concentradas en el binario principal `main.rs`.

Compilación cruzada con `.cargo/config.toml`

El archivo `.cargo/config.toml` define la configuración específica de compilación para el microcontrolador. En un ordenador convencional, Rust compila por defecto para la arquitectura de la propia máquina. Sin embargo, en este proyecto el programa debe ejecutarse sobre un ARM Cortex-M4F, por lo que es necesario realizar compilación cruzada.

Para ello se selecciona el target:

```
thumbv7em-none-eabihf
```

Este target corresponde a microcontroladores ARM Cortex-M4 y Cortex-M7 con unidad de coma flotante hardware. La parte `none` indica que no existe sistema operativo, y por tanto el programa debe construirse como firmware autónomo.

El mismo archivo configura también el *runner* empleado por Cargo. Gracias a esta configuración, el comando:

```
cargo run --release --bin main
```

no solo compila el firmware, sino que además invoca **probe-rs** para cargarlo directamente en la placa STM32G474RE. Esto simplifica el ciclo de desarrollo, ya que permite compilar, flashear y ejecutar desde una única orden.

También se especifican opciones de enlazado, como el uso de **flip-link** y de los scripts **link.x** y **defmt.x**. Estos elementos son necesarios para situar correctamente el programa en memoria y para habilitar el sistema de trazas usado por **defmt**.

Mapa de memoria con **memory.x**

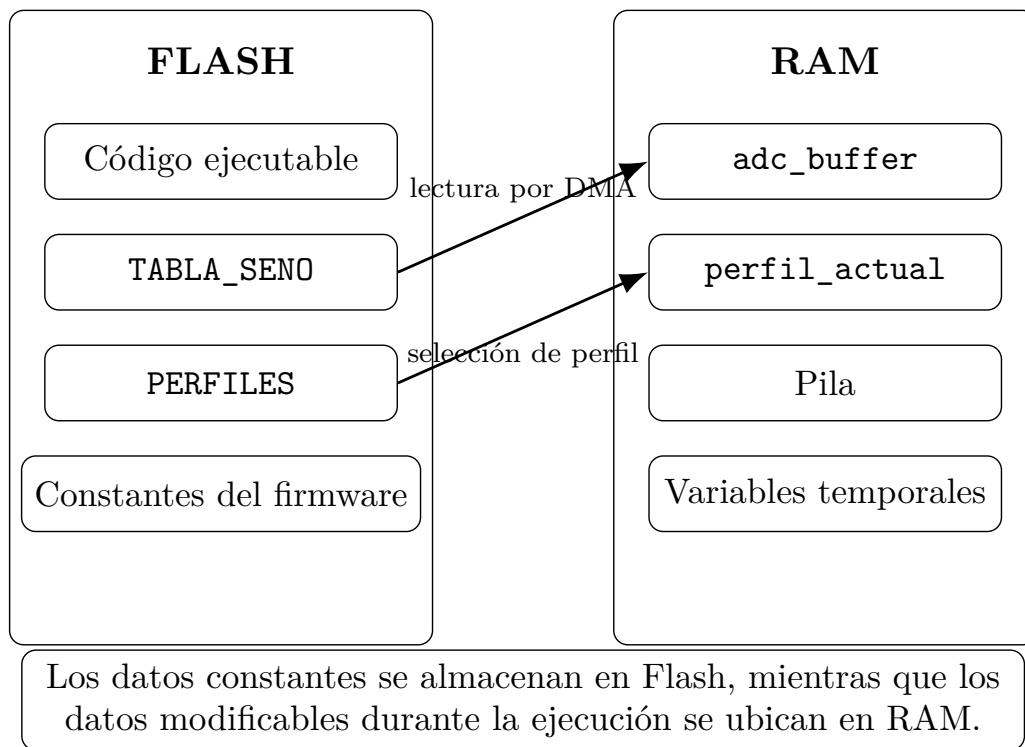


Figura 6: Distribución conceptual de los elementos principales del firmware entre memoria Flash y RAM.

3.2.5. Ejecución del firmware y organización de memoria

```

$ ./data/analysis/analysis.py --freqs=[0.001, 0.002, 0.003, 0.004, 0.005, 0.006, 0.007, 0.008, 0.009, 0.01, 0.011, 0.012, 0.013, 0.014, 0.015, 0.016, 0.017, 0.018, 0.019, 0.02, 0.021, 0.022, 0.023, 0.024, 0.025, 0.026, 0.027, 0.028, 0.029, 0.03, 0.031, 0.032, 0.033, 0.034, 0.035, 0.036, 0.037, 0.038, 0.039, 0.04, 0.041, 0.042, 0.043, 0.044, 0.045, 0.046, 0.047, 0.048, 0.049, 0.05, 0.051, 0.052, 0.053, 0.054, 0.055, 0.056, 0.057, 0.058, 0.059, 0.06, 0.061, 0.062, 0.063, 0.064, 0.065, 0.066, 0.067, 0.068, 0.069, 0.07, 0.071, 0.072, 0.073, 0.074, 0.075, 0.076, 0.077, 0.078, 0.079, 0.08, 0.081, 0.082, 0.083, 0.084, 0.085, 0.086, 0.087, 0.088, 0.089, 0.09, 0.091, 0.092, 0.093, 0.094, 0.095, 0.096, 0.097, 0.098, 0.099, 0.1, 0.101, 0.102, 0.103, 0.104, 0.105, 0.106, 0.107, 0.108, 0.109, 0.11, 0.111, 0.112, 0.113, 0.114, 0.115, 0.116, 0.117, 0.118, 0.119, 0.12, 0.121, 0.122, 0.123, 0.124, 0.125, 0.126, 0.127, 0.128, 0.129, 0.13, 0.131, 0.132, 0.133, 0.134, 0.135, 0.136, 0.137, 0.138, 0.139, 0.14, 0.141, 0.142, 0.143, 0.144, 0.145, 0.146, 0.147, 0.148, 0.149, 0.15, 0.151, 0.152, 0.153, 0.154, 0.155, 0.156, 0.157, 0.158, 0.159, 0.16, 0.161, 0.162, 0.163, 0.164, 0.165, 0.166, 0.167, 0.168, 0.169, 0.17, 0.171, 0.172, 0.173, 0.174, 0.175, 0.176, 0.177, 0.178, 0.179, 0.18, 0.181, 0.182, 0.183, 0.184, 0.185, 0.186, 0.187, 0.188, 0.189, 0.19, 0.191, 0.192, 0.193, 0.194, 0.195, 0.196, 0.197, 0.198, 0.199, 0.2, 0.201, 0.202, 0.203, 0.204, 0.205, 0.206, 0.207, 0.208, 0.209, 0.21, 0.211, 0.212, 0.213, 0.214, 0.215, 0.216, 0.217, 0.218, 0.219, 0.22, 0.221, 0.222, 0.223, 0.224, 0.225, 0.226, 0.227, 0.228, 0.229, 0.23, 0.231, 0.232, 0.233, 0.234, 0.235, 0.236, 0.237, 0.238, 0.239, 0.24, 0.241, 0.242, 0.243, 0.244, 0.245, 0.246, 0.247, 0.248, 0.249, 0.25, 0.251, 0.252, 0.253, 0.254, 0.255, 0.256, 0.257, 0.258, 0.259, 0.26, 0.261, 0.262, 0.263, 0.264, 0.265, 0.266, 0.267, 0.268, 0.269, 0.27, 0.271, 0.272, 0.273, 0.274, 0.275, 0.276, 0.277, 0.278, 0.279, 0.28, 0.281, 0.282, 0.283, 0.284, 0.285, 0.286, 0.287, 0.288, 0.289, 0.29, 0.291, 0.292, 0.293, 0.294, 0.295, 0.296, 0.297, 0.298, 0.299, 0.3, 0.301, 0.302, 0.303, 0.304, 0.305, 0.306, 0.307, 0.308, 0.309, 0.31, 0.311, 0.312, 0.313, 0.314, 0.315, 0.316, 0.317, 0.318, 0.319, 0.32, 0.321, 0.322, 0.323, 0.324, 0.325, 0.326, 0.327, 0.328, 0.329, 0.33, 0.331, 0.332, 0.333, 0.334, 0.335, 0.336, 0.337, 0.338, 0.339, 0.34, 0.341, 0.342, 0.343, 0.344, 0.345, 0.346, 0.347, 0.348, 0.349, 0.35, 0.351, 0.352, 0.353, 0.354, 0.355, 0.356, 0.357, 0.358, 0.359, 0.36, 0.361, 0.362, 0.363, 0.364, 0.365, 0.366, 0.367, 0.368, 0.369, 0.37, 0.371, 0.372, 0.373, 0.374, 0.375, 0.376, 0.377, 0.378, 0.379, 0.38, 0.381, 0.382, 0.383, 0.384, 0.385, 0.386, 0.387, 0.388, 0.389, 0.39, 0.391, 0.392, 0.393, 0.394, 0.395, 0.396, 0.397, 0.398, 0.399, 0.4, 0.401, 0.402, 0.403, 0.404, 0.405, 0.406, 0.407, 0.408, 0.409, 0.41, 0.411, 0.412, 0.413, 0.414, 0.415, 0.416, 0.417, 0.418, 0.419, 0.42, 0.421, 0.422, 0.423, 0.424, 0.425, 0.426, 0.427, 0.428, 0.429, 0.43, 0.431, 0.432, 0.433, 0.434, 0.435, 0.436, 0.437, 0.438, 0.439, 0.44, 0.441, 0.442, 0.443, 0.444, 0.445, 0.446, 0.447, 0.448, 0.449, 0.45, 0.451, 0.452, 0.453, 0.454, 0.455, 0.456, 0.457, 0.458, 0.459, 0.46, 0.461, 0.462, 0.463, 0.464, 0.465, 0.466, 0.467, 0.468, 0.469, 0.47, 0.471, 0.472, 0.473, 0.474, 0.475, 0.476, 0.477, 0.478, 0.479, 0.48, 0.481, 0.482, 0.483, 0.484, 0.485, 0.486, 0.487, 0.488, 0.489, 0.49, 0.491, 0.492, 0.493, 0.494, 0.495, 0.496, 0.497, 0.498, 0.499, 0.5, 0.501, 0.502, 0.503, 0.504, 0.505, 0.506, 0.507, 0.508, 0.509, 0.51, 0.511, 0.512, 0.513, 0.514, 0.515, 0.516, 0.517, 0.518, 0.519, 0.52, 0.521, 0.522, 0.523, 0.524, 0.525, 0.526, 0.527, 0.528, 0.529, 0.53, 0.531, 0.532, 0.533, 0.534, 0.535, 0.536, 0.537, 0.538, 0.539, 0.54, 0.541, 0.542, 0.543, 0.544, 0.545, 0.546, 0.547, 0.548, 0.549, 0.55, 0.551, 0.552, 0.553, 0.554, 0.555, 0.556, 0.557, 0.558, 0.559, 0.56, 0.561, 0.562, 0.563, 0.564, 0.565, 0.566, 0.567, 0.568, 0.569, 0.57, 0.571, 0.572, 0.573, 0.574, 0.575, 0.576, 0.577, 0.578, 0.579, 0.58, 0.581, 0.582, 0.583, 0.584, 0.585, 0.586, 0.587, 0.588, 0.589, 0.59, 0.591, 0.592, 0
```

Figura 7: Salida por consola durante la ejecución del firmware, mostrando la frecuencia activa y las muestras adquiridas por el ADC.

El archivo `memory.x` define las regiones de memoria disponibles para el firmware. En un sistema embebido, el enlazador necesita conocer la dirección de inicio y el tamaño de la memoria Flash y de la memoria RAM del microcontrolador.

La memoria Flash se utiliza principalmente para almacenar el código del programa y las constantes, mientras que la RAM contiene variables globales modificables, pila, buffers y datos temporales de ejecución. En este proyecto, el buffer utilizado por el DMA para almacenar muestras del ADC depende directamente de esta configuración de memoria.

Además de los buffers utilizados por DMA, el firmware emplea memoria no volátil para almacenar tablas constantes de funcionamiento. La tabla senoidal y los perfiles de frecuencia se declaran como constantes, por lo que el compilador los ubica en la región de memoria Flash. Esto permite disponer de varias configuraciones predefinidas sin ocupar RAM dinámica ni requerir asignación en tiempo de ejecución, algo especialmente adecuado en un entorno *bare-metal* y `no_std`.

Un ejemplo simplificado de este archivo es:

MEMORY

```
{
  FLASH : ORIGIN = 0x08000000 , LENGTH = 512K
  RAM   : ORIGIN = 0x20000000 , LENGTH = 96K
}
```

En este proyecto, la memoria Flash contiene tanto el código ejecutable como los datos constantes del sistema, entre ellos la tabla de muestras de la señal senoidal y la tabla de perfiles de frecuencia. Por otro lado, la RAM se reserva para elementos modificables durante la ejecución, como el índice del perfil activo y el búfer circular utilizado por el DMA para almacenar las muestras procedentes del ADC.

Esta configuración debe coincidir con las características reales del microcontrolador utilizado. Una definición incorrecta del mapa de memoria puede provocar errores de enlazado, desaprovechamiento de memoria disponible o fallos en tiempo de ejecución si el programa accede a regiones no válidas.

Código común de arranque y errores en `src/lib.rs`

El archivo `src/lib.rs` contiene parte de la infraestructura común utilizada por los distintos binarios del proyecto. En él se integran elementos como el sistema de logging global, el comportamiento ante pánicos y el tratamiento básico de excepciones.

En aplicaciones Rust convencionales, la biblioteca estándar proporciona mecanismos de gestión de errores, impresión por consola y finalización del programa. En un sistema *no_std*, estas funcionalidades no existen por defecto, por lo que deben definirse explícitamente.

En este proyecto se emplea `panic-probe` junto con `defmt` para poder observar errores durante la ejecución mediante la sonda de depuración. Esto facilita detectar fallos en fases tempranas del desarrollo, especialmente cuando el programa se ejecuta sin pantalla, teclado ni sistema operativo.

Estructura de binarios de prueba

Durante el desarrollo se emplearon pruebas independientes para validar cada periférico por separado antes de realizar la integración completa. Esta estrategia permitió reducir el riesgo técnico antes de concentrar la funcionalidad en el firmware principal.

Por ejemplo, se desarrollaron pruebas específicas para:

- Comprobar el entorno mínimo de ejecución y depuración.
- Configurar un pin GPIO como salida.
- Validar la lectura del ADC en el pin PA0.
- Generar una tensión analógica mediante el DAC en PA4.
- Integrar generación y adquisición en un mismo flujo.

Esta organización redujo la complejidad durante la depuración. En lugar de probar directamente todo el sistema completo, cada bloque funcional se verificó de forma incremental. Posteriormente, las partes validadas se integraron en el binario principal `main.rs`.

Depuración con `defmt` y `probe-rs`

La depuración de firmware embebido presenta dificultades adicionales respecto al software convencional, ya que no existe una salida estándar por consola. Para resolver este problema se utilizó `defmt`, un sistema de logging diseñado específicamente para microcontroladores.

Las trazas generadas por el firmware se transmiten mediante RTT y son recogidas en el ordenador de desarrollo a través de `probe-rs`. Esto permite observar mensajes de estado, valores leídos del ADC, avisos de desviación y eventos relevantes del sistema sin necesidad de implementar inicialmente una comunicación UART propia.

Este mecanismo resultó especialmente útil durante la fase de pruebas, ya que permitió comprobar si los periféricos estaban siendo inicializados correctamente y si el buffer de muestras del ADC recibía datos durante la ejecución.

Importancia de la infraestructura en el proyecto

La infraestructura descrita no constituye un elemento auxiliar, sino una parte fundamental del proyecto. Sin ella, el código Rust no podría ejecutarse correctamente sobre el STM32G474RE. La configuración de compilación cruzada, el mapa de memoria, el sistema de arranque, los perfiles

de optimización, la carga mediante `probe-rs` y la depuración mediante `defmt` forman la base sobre la que se construye el firmware de generación y adquisición de señal.

Por tanto, el desarrollo del sistema no se limitó únicamente a programar los periféricos ADC, DAC, DMA y TIM6, sino que también incluyó la creación de un entorno completo de desarrollo embebido en Rust, reproducible y adaptado a la placa utilizada.

3.2.6. Instrumentación de laboratorio

Para la validación del sistema y el análisis de las señales generadas y adquiridas, se empleó una **Analog Discovery 3** de Digilent proporcionada por el departamento. Este equipo permitió observar en tiempo real tanto la señal senoidal generada por el convertidor digital-analógico (DAC) como la señal capturada mediante el convertidor analógico-digital (ADC), facilitando la verificación del correcto funcionamiento del sistema.

Durante el desarrollo se utilizaron las entradas analógicas del equipo y el software WaveForms para visualizar y exportar las señales medidas. La instrumentación se empleó principalmente en las fases de prueba del generador de señal, calibración del sistema de adquisición y verificación del comportamiento bajo carga conocida.

El uso de este instrumento resultó fundamental para comprobar la integridad de la forma de onda generada, detectar posibles distorsiones o errores de sincronización y contrastar experimentalmente las muestras observadas mediante las trazas del firmware.

3.2.7. Otros componentes considerados

Durante la fase de diseño, se evaluó la posibilidad de utilizar integrados externos para la conversión de señal (DAC/ADC de mayor resolución). Sin embargo, tras analizar las especificaciones del **STM32G474RE**, se determinó que sus convertidores internos de 12 bits y su alta velocidad de muestreo eran más que suficientes para aplicaciones biomédicas de este rango, cumpliendo así con el requisito **R1** de maximizar la integración.

3.3. Análisis de costes estimados

El desarrollo del presente proyecto implica distintos tipos de costes que van más allá del material estrictamente utilizado. Para ofrecer una visión más realista del esfuerzo necesario en un contexto profesional, se distinguen dos categorías principales: costes de materiales y costes derivados del trabajo de ingeniería. Las cifras presentadas constituyen estimaciones orientativas con fines académicos.

Costes de materiales

Componente	Coste estimado
Tarjeta Nucleo-G474RE	25.00 €
Componentes pasivos y placa perforada	5.00 €
Cableado, conectores y consumibles de laboratorio	15.00 €
Software (Rust, LaTeX, WaveForms y herramientas Open Source)	0.00 €
Subtotal materiales	45.00 €

Tabla 1: Costes asociados a materiales utilizados en el proyecto

Equipamiento de laboratorio

La instrumentación utilizada durante las pruebas, en concreto la **Analog Discovery 3** de Digilent, fue proporcionada por el departamento. Por tanto, no supuso un coste de adquisición para el proyecto y no se incluye dentro del presupuesto estimado.

Costes de ingeniería

El desarrollo del firmware, diseño electrónico, pruebas experimentales, documentación y elaboración de la memoria implicaron una dedicación estimada de 310 horas de trabajo técnico, coherente con la planificación de hitos descrita posteriormente. Para efectos comparativos con un entorno profesional, se considera una tarifa de 30 € por hora.

Concepto	Coste estimado
Trabajo de ingeniería (310 h × 30 €/h)	9300.00 €
Subtotal ingeniería	9300.00 €

Tabla 2: Estimación del coste de desarrollo técnico

Considerando el conjunto de categorías anteriores, el coste total estimado del proyecto asciende a:

Coste total estimado: 9345.00 €

3.4. Planificación de hitos

El proyecto se ha dividido en cinco hitos principales. La planificación parte del día **2 de marzo de 2026** y finaliza el día **26 de mayo de 2026**. Se organiza considerando jornadas de trabajo de aproximadamente **5 horas**, con una dedicación total estimada de **310 horas**.

La distribución temporal se ha organizado de forma secuencial: primero se reserva un bloque amplio para el aprendizaje de Rust embebido y la configuración del entorno, posteriormente se desarrolla la generación de señal mediante DAC, después la adquisición mediante ADC, y finalmente se realizan las pruebas experimentales con la Analog Discovery 3 antes de la redacción final de la memoria.

3.4.1. H1 – Aprendizaje y entorno Rust embebido funcional

Código	Tarea	Inicio	Fin	Horas
T1.1	Estudio inicial de Rust	02/03/2026	04/03/2026	15
T1.2	Rust embebido: <i>no_std</i> y <i>bare-metal</i>	05/03/2026	09/03/2026	15
T1.3	Ecosistema: <i>cortex-m</i> , PAC, <i>defmt</i> y <i>probe-rs</i>	10/03/2026	12/03/2026	15
T1.4	Instalación y configuración de herramientas	13/03/2026	16/03/2026	10
T1.5	Estructura del proyecto y <i>Cargo.toml</i>	17/03/2026	18/03/2026	10
T1.6	Mapa de memoria y enlazado	19/03/2026	20/03/2026	10
T1.7	Primer firmware mínimo	23/03/2026	23/03/2026	5
T1.8	Depuración con <i>defmt</i> y <i>probe-rs</i>	24/03/2026	24/03/2026	5
Subtotal H1				85

Tabla 3: Planificación del hito H1

3.4.2. H2 – Generación de señal senoidal mediante DAC

Código	Tarea	Inicio	Fin	Horas
T2.1	Estudio de DAC, TIM6, DMA y DMAMUX	25/03/2026	26/03/2026	10
T2.2	Configuración de relojes y GPIOA	27/03/2026	27/03/2026	5
T2.3	Tabla senoidal de 32 muestras	30/03/2026	31/03/2026	10
T2.4	Configuración de TIM6 como TRGO	01/04/2026	02/04/2026	10
T2.5	Configuración del DAC	03/04/2026	07/04/2026	15
T2.6	DMA circular hacia DAC	08/04/2026	09/04/2026	10
T2.7	Perfiles de frecuencia	10/04/2026	10/04/2026	5
T2.8	Pruebas preliminares mediante defmt	13/04/2026	13/04/2026	5
Subtotal H2				70

Tabla 4: Planificación del hito H2

3.4.3. H3 – Adquisición de señal mediante ADC

Código	Tarea	Inicio	Fin	Horas
T3.1	Estudio del ADC y calibración	14/04/2026	15/04/2026	10
T3.2	Configuración de PA0 y canal ADC	16/04/2026	16/04/2026	5
T3.3	Reloj, regulador y calibración del ADC	17/04/2026	20/04/2026	10
T3.4	Disparo externo mediante TIM6	21/04/2026	22/04/2026	10
T3.5	DMA circular para búfer ADC	23/04/2026	24/04/2026	10
T3.6	Integración DAC-ADC-TIM6-DMA	27/04/2026	28/04/2026	10
T3.7	Lectura del búfer y trazas defmt	29/04/2026	29/04/2026	5
Subtotal H3				60

Tabla 5: Planificación del hito H3

3.4.4. H4 – Validación experimental con Analog Discovery 3

Código	Tarea	Inicio	Fin	Horas
T4.1	Preparación del entorno de pruebas	30/04/2026	30/04/2026	5
T4.2	Plan de pruebas experimentales	01/05/2026	01/05/2026	5
T4.3	Medida de señal DAC con Analog Discovery 3	04/05/2026	05/05/2026	10
T4.4	Pruebas con cargas conocidas	06/05/2026	07/05/2026	10
T4.5	Prueba de estabilidad	08/05/2026	08/05/2026	5
T4.6	Análisis y capturas de resultados	11/05/2026	12/05/2026	10
Subtotal H4				45

Tabla 6: Planificación del hito H4

3.4.5. H5 – Documentación y cierre del proyecto

Código	Tarea	Inicio	Fin	Horas
T5.1	Estructura final, resumen e índice	13/05/2026	13/05/2026	5
T5.2	Introducción, objetivos y descripción	14/05/2026	14/05/2026	5
T5.3	Estado del arte y fundamentos	15/05/2026	18/05/2026	10
T5.4	Planificación, requisitos y costes	19/05/2026	20/05/2026	10
T5.5	Desarrollo técnico y firmware	21/05/2026	21/05/2026	5
T5.6	Pruebas, resultados y conclusiones	22/05/2026	22/05/2026	5
T5.7	Bibliografía, anexos y revisión final	25/05/2026	26/05/2026	10
Subtotal H5				50

Tabla 7: Planificación del hito H5

3.4.6. Resumen temporal

Hito	Inicio	Fin	Horas
H1 – Aprendizaje y entorno Rust embebido	02/03/2026	24/03/2026	85
H2 – Generación de señal mediante DAC	25/03/2026	13/04/2026	70
H3 – Adquisición de señal mediante ADC	14/04/2026	29/04/2026	60
H4 – Validación experimental con Analog Discovery 3	30/04/2026	12/05/2026	45
H5 – Documentación y cierre del proyecto	13/05/2026	26/05/2026	50
Total	02/03/2026	26/05/2026	310

Tabla 8: Resumen de la planificación temporal del proyecto

3.4.7. Diagrama de Gantt

En la Figura 8 se representa la planificación temporal del proyecto dividida por hitos. El calendario comienza el 2 de marzo de 2026 y finaliza el 26 de mayo de 2026, con una dedicación total estimada de 310 horas.

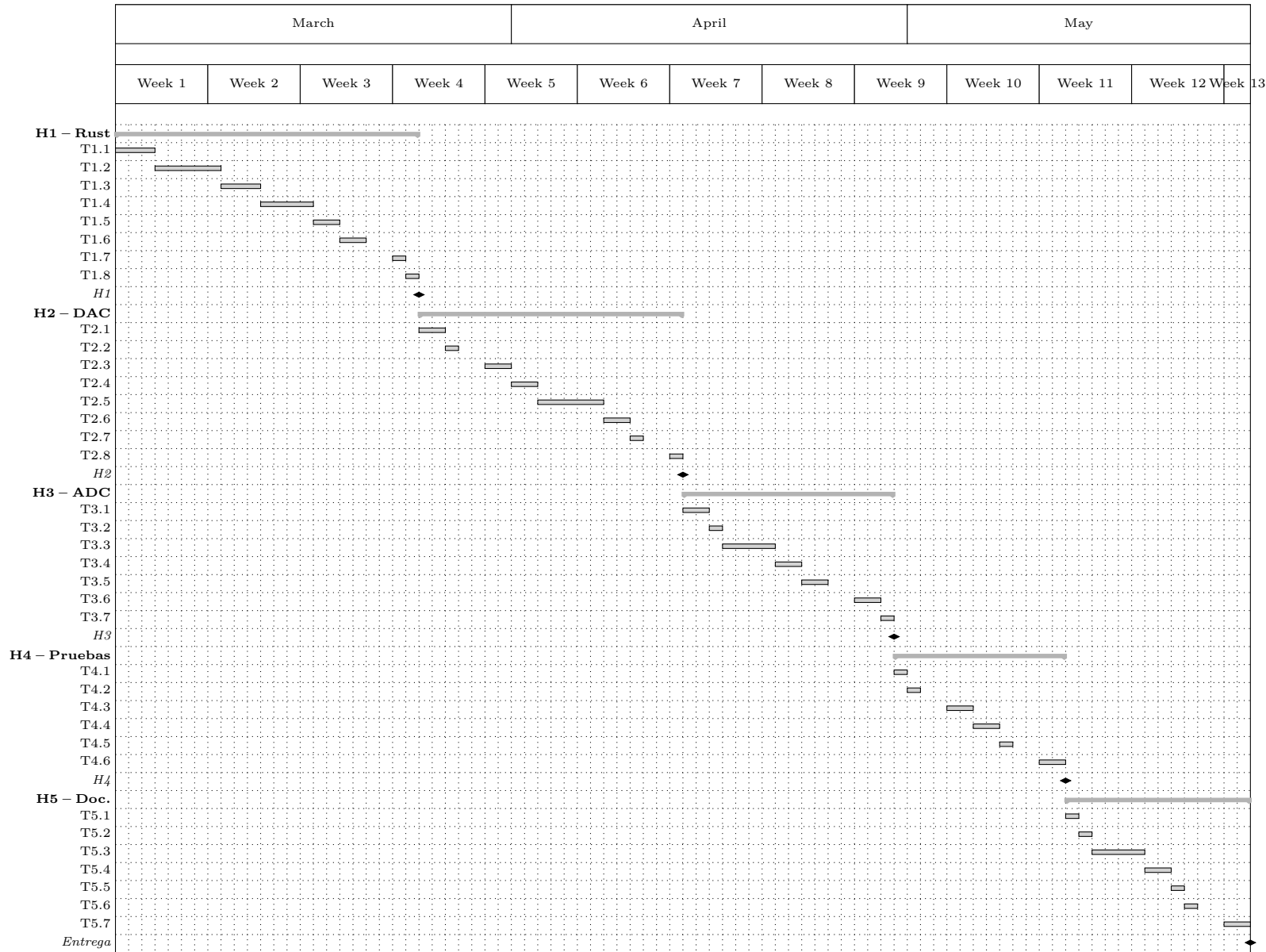


Figura 8: Diagrama de Gantt del proyecto dividido por hitos

4. Desarrollo del proyecto

4.1. Fundamentos del Sistema Microcontrolador

El sistema se apoya en una arquitectura de 32 bits. La clave del rendimiento reside en la interconexión de periféricos mediante el bus **AHB**, permitiendo que el DMA mueva datos entre el DAC/ADC y la memoria RAM sin intervención del núcleo ARM Cortex-M4, liberando a este para tareas de procesamiento de datos.

4.2. Diseño del sistema

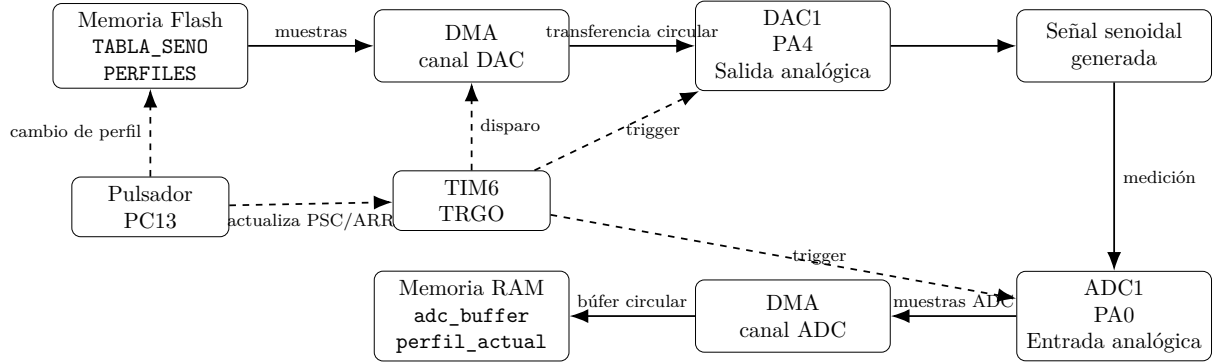


Figura 9: Arquitectura general del sistema implementado, mostrando la generación mediante DAC, la adquisición mediante ADC y la sincronización común mediante TIM6.

4.2.1. Funcionamiento integrado

La generación se basa en una técnica de síntesis digital directa (DDS) simplificada. Una tabla de 32 valores define un ciclo completo de la onda, y el DAC se actualiza mediante eventos de disparo generados por TIM6. Por tanto, la frecuencia de salida depende del reloj de TIM6 y de los divisores enteros configurados:

$$f_{out} = \frac{f_{TIM6}}{(PSC + 1)(ARR + 1)N_{muestras}}$$

El firmware no modifica la configuración global del reloj después del arranque. Las medidas realizadas con el osciloscopio son coherentes con un reloj de TIM6 próximo a 16 MHz. Debido a la cuantización introducida por *PSC* y *ARR*, las frecuencias obtenidas no coinciden exactamente con los valores inicialmente utilizados como etiquetas.

El mismo disparo hardware se utiliza para coordinar la adquisición mediante **ADC1**. En lugar de ejecutar un bucle software que escriba cada muestra en el DAC o lea cada conversión del ADC, el firmware configura los periféricos para que TIM6 marque el ritmo y DMA realice las transferencias de datos. La CPU queda reservada principalmente para la inicialización, la selección de perfiles de frecuencia y la monitorización de las muestras adquiridas.

Los perfiles de frecuencia modifican los valores *PSC* y *ARR* de TIM6, permitiendo cambiar la frecuencia de generación y adquisición sin alterar la tabla senoidal ni recompilar el programa.

4.3. Desarrollo del sistema (Fases)

Las fases F1–F8 describen la evolución técnica seguida durante la implementación y validación del sistema. No sustituyen a los hitos temporales H1–H5 de la planificación, sino que los detallan desde el punto de vista del trabajo realizado: aprendizaje e infraestructura, prototipos de periféricos, integración, selección de perfiles y validación experimental.

4.3.1. F1: Planteamiento y estudio inicial

En esta fase se definió el alcance general del proyecto y se realizó un estudio preliminar de su viabilidad técnica. Se analizaron los objetivos principales del trabajo, los requisitos funcionales del sistema y las restricciones impuestas al diseño, tanto a nivel de implementación embebida como de integración hardware. También se llevó a cabo una primera evaluación de la familia STM32G4, prestando especial atención a sus capacidades analógicas internas. Este estudio permitió comprobar que el microcontrolador STM32G474RE disponía de periféricos adecuados para el desarrollo del proyecto, como DAC, ADC, temporizadores y DMA. A partir de este análisis se justificó la elección de esta plataforma como base del sistema.

4.3.2. F2: Aprendizaje de Rust embebido e infraestructura

Durante esta etapa se adquirieron los conocimientos necesarios para el desarrollo de aplicaciones embebidas en Rust. Se estudió el ecosistema `embedded-hal`, la organización de los paquetes de soporte para microcontroladores STM32 y las herramientas empleadas para la compilación, carga y depuración del firmware. Asimismo, se configuró la infraestructura básica de desarrollo, incluyendo el entorno de compilación para arquitecturas ARM Cortex-M, el uso de `probe-rs` para la programación y depuración del microcontrolador, y la estructura inicial del proyecto en Rust. Esta fase resultó fundamental para establecer una base de trabajo estable sobre la que implementar posteriormente los distintos periféricos. Dado que el lenguaje C continúa siendo la opción predominante en el ámbito de los sistemas embebidos, se detectó que el ecosistema Rust para la familia STM32G4 presenta todavía ciertas limitaciones. En particular, algunas capas de abstracción hardware disponibles se encuentran en fases tempranas de desarrollo o no ofrecen soporte completo para todos los periféricos requeridos. Por ello, fue necesario trabajar en algunos casos a un nivel más próximo a los registros del microcontrolador.

4.3.3. F3: Estudio del STM32G474RE y periféricos analógicos

En esta fase se profundizó en el estudio del microcontrolador STM32G474RE y de los periféricos necesarios para el proyecto: DAC, ADC, DMA y temporizadores. También se estudiaron los registros específicos asociados a cada bloque, ya que el soporte de alto nivel disponible en Rust no cubría completamente todas las funcionalidades necesarias.

Este análisis permitió definir la arquitectura interna del firmware: generación mediante DAC, temporización mediante TIM6, transferencia automática por DMA y adquisición mediante ADC.

4.3.4. F4: Prototipos mínimos de GPIO, DAC y ADC

Una vez configurado el entorno de desarrollo y estudiados los periféricos principales, se implementaron prototipos mínimos para validar el funcionamiento básico del microcontrolador. Inicialmente se realizaron pruebas con GPIO, con el objetivo de comprobar la correcta compilación, carga y ejecución del firmware sobre la placa de desarrollo. Posteriormente, se desarrollaron pruebas independientes para el DAC y el ADC. En el caso del DAC, se verificó la capacidad de generar valores analógicos simples a partir de datos digitales definidos por software. En el caso del ADC, se comprobó la lectura de señales analógicas y se implementó una rutina de calibración inicial para compensar el error de offset interno del chip. Estos prototipos permitieron validar de forma incremental cada bloque funcional antes de abordar su integración completa. Además, facilitaron la detección temprana de errores de configuración y permitieron familiarizarse con el acceso a los registros del STM32G474RE desde Rust.

4.3.5. F5: Integración DAC-DMA-TIM6

En esta fase se integraron el DAC, el controlador DMA y el temporizador TIM6 para implementar la generación automática de señal. Se definió una tabla senoidal en memoria y se

configuró el DMA para transferir sus muestras al DAC siguiendo el ritmo marcado por TIM6. Esta etapa permitió pasar de pruebas manuales de salida analógica a una generación periódica y autónoma.

4.3.6. F6: Integración ADC-DMA y lectura de muestras

Durante esta etapa se configuró el ADC junto con el DMA para realizar la adquisición automática de muestras analógicas. Se ajustó la calibración inicial, el canal de entrada y el modo de conversión, y se configuró un búfer de muestras en memoria para recibir los resultados. Con ello se cerró el ciclo de generación y adquisición dentro del propio microcontrolador.

4.3.7. F7: Selección de perfiles de frecuencia mediante pulsador

Una vez validado el funcionamiento conjunto del DAC, ADC, DMA y TIM6, se incorporó la selección de distintas frecuencias sin recompilar el firmware. Cada perfil almacena los valores de prescaler (PSC) y auto-reload (ARR) de TIM6, y el pulsador de usuario conectado a PC13 permite recorrerlos durante la ejecución. Esta ampliación facilita realizar ensayos a distintas frecuencias de excitación manteniendo la sincronización entre generación y adquisición.

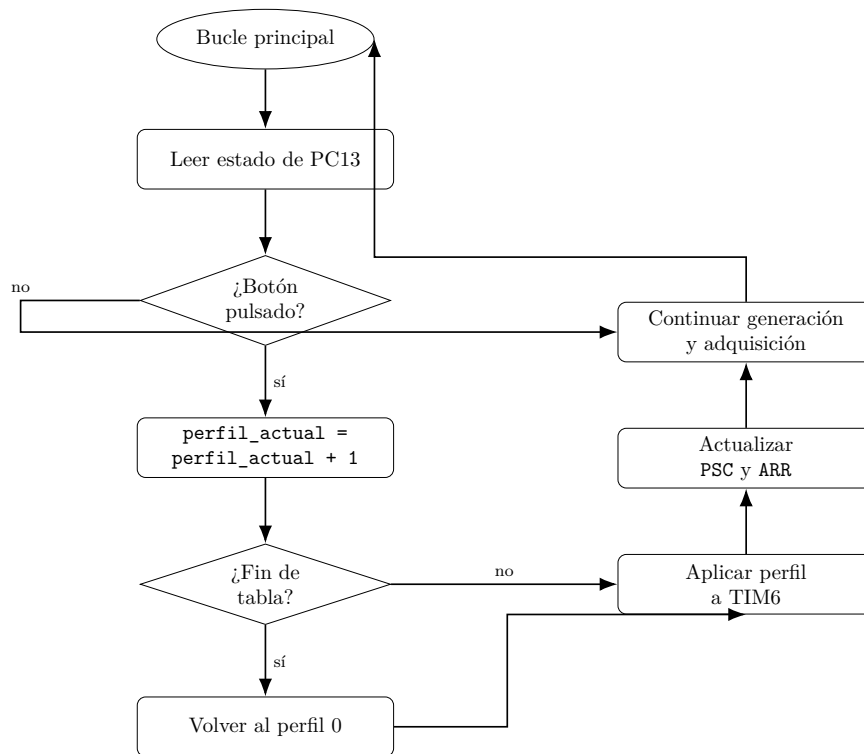


Figura 10: Flujo de selección cíclica de perfiles de frecuencia mediante el pulsador de usuario conectado a PC13.

Perfil	Frecuencia medida	PSC	ARR
0	4.72 Hz	169	624
1	9.42 Hz	169	312
2	23.60 Hz	169	124
3	46.83 Hz	169	62
4	95.15 Hz	169	30
5	295.00 Hz	169	9
6	2697.08 Hz	5	30
7	5225.39 Hz	5	15
8	20896.21 Hz	5	3
9	41798.46 Hz	5	1

Tabla 9: Perfiles de frecuencia definidos en el firmware.

El campo entero `frecuencia_hz` utilizado en los mensajes de depuración almacena estos valores redondeados al hercio más cercano. Los valores con decimales de la tabla proceden de la estimación realizada sobre las capturas del osciloscopio.

4.3.8. F8: Validación experimental con carga resistiva-capacitiva

Finalmente, se llevó a cabo la validación experimental del sistema completo. Para ello, se comparó la señal adquirida por el ADC con la señal observada mediante la **Analog Discovery 3**, comprobando la coherencia entre ambas medidas y verificando el correcto funcionamiento de la cadena de generación y adquisición.

Para comprobar el comportamiento del sistema ante una carga de impedancia conocida se montó un circuito de prueba formado por tres resistencias de $10\text{ k}\Omega$ y un condensador de 10 nF . La resistencia R_1 se conectó entre la entrada V_A y el punto de medida V_B . Desde V_B hasta masa se dispusieron dos ramas en paralelo: una formada por R_2 y R_3 en serie, con una resistencia equivalente de $20\text{ k}\Omega$, y otra formada por el condensador. De esta forma se obtuvo una carga con comportamiento dependiente de la frecuencia.

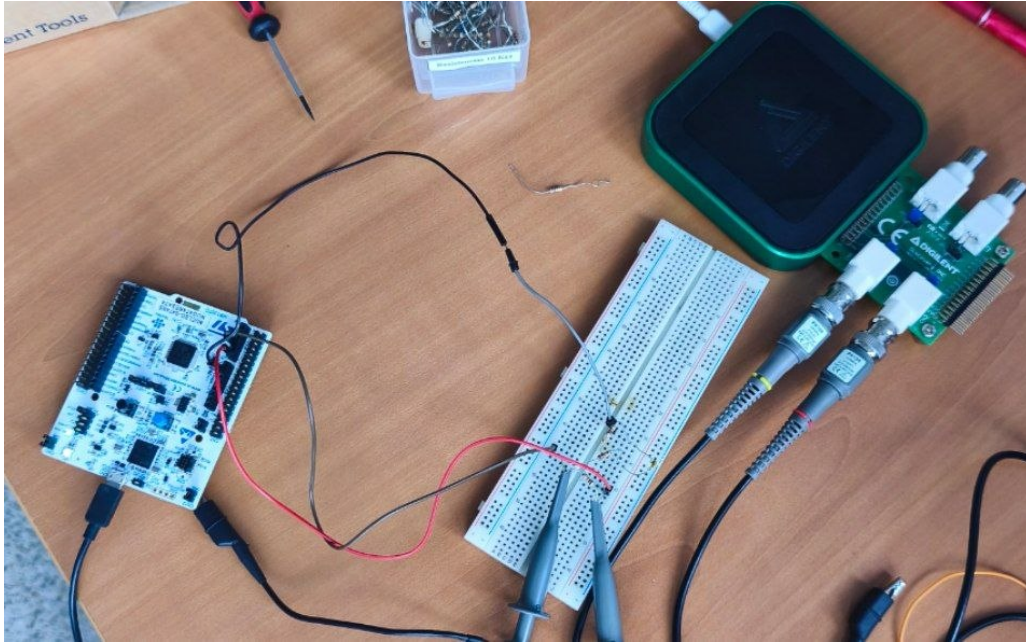


Figura 11: Montaje experimental utilizado durante las pruebas con la Analog Discovery 3 de Digilent.

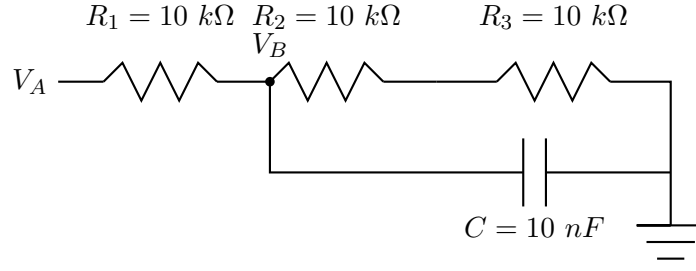


Figura 12: Circuito de prueba utilizado para las medidas de impedancia.

Formas de onda observadas

Antes de aplicar el modelo eléctrico, se representaron las formas de onda registradas en cada ensayo. En las Figuras 13, 14 y 15 se muestran dos periodos de los canales V_A y V_B . Para facilitar la comparación se eliminó únicamente el valor medio de cada canal y se mantuvo una escala vertical común de $\pm 1,8$ V. Por tanto, las gráficas conservan tanto la relación de amplitudes como el desplazamiento temporal entre ambas señales.

El eje horizontal se expresa en ciclos de la señal y no en segundos, lo que permite comparar en una misma escala capturas realizadas desde décimas de hercio hasta decenas de kilohercios. Sobre cada caso se indican la frecuencia medida, la ganancia $|V_B/V_A|$ y el desfase calculado a partir de la captura completa.

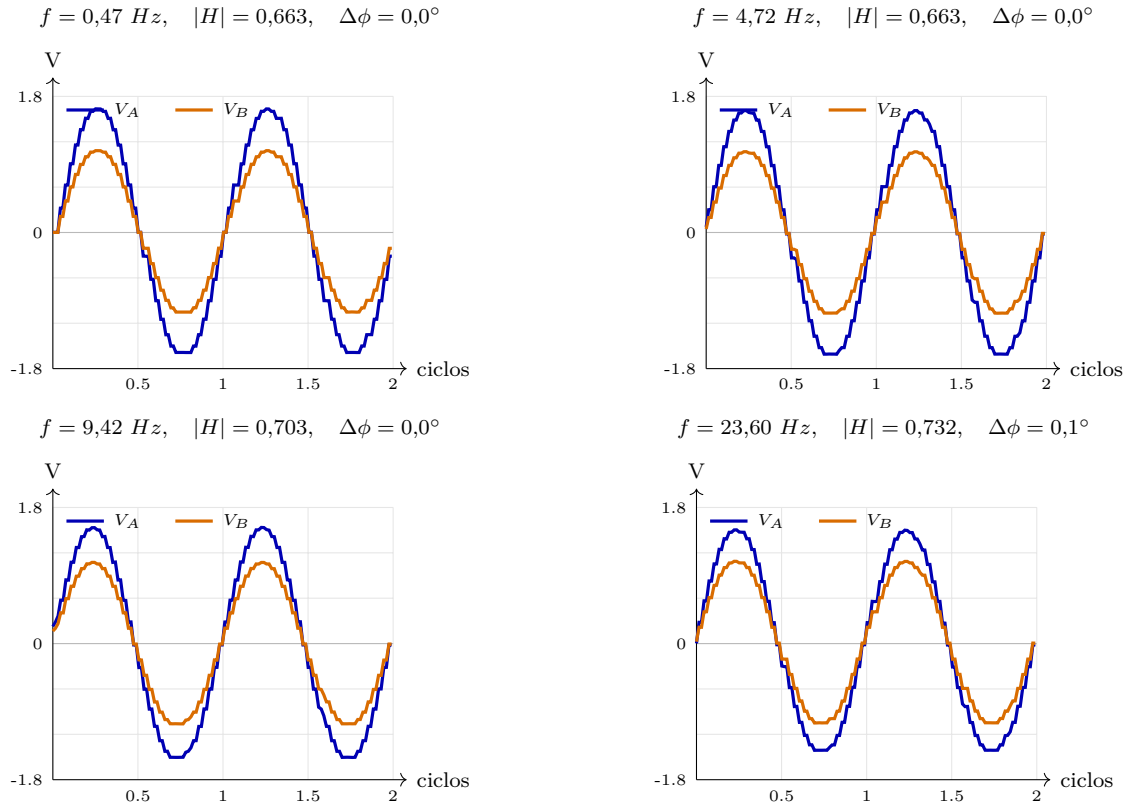


Figura 13: Formas de onda medidas a baja frecuencia. Se representan las componentes alternas de V_A y V_B durante dos periodos, manteniendo la misma escala vertical.

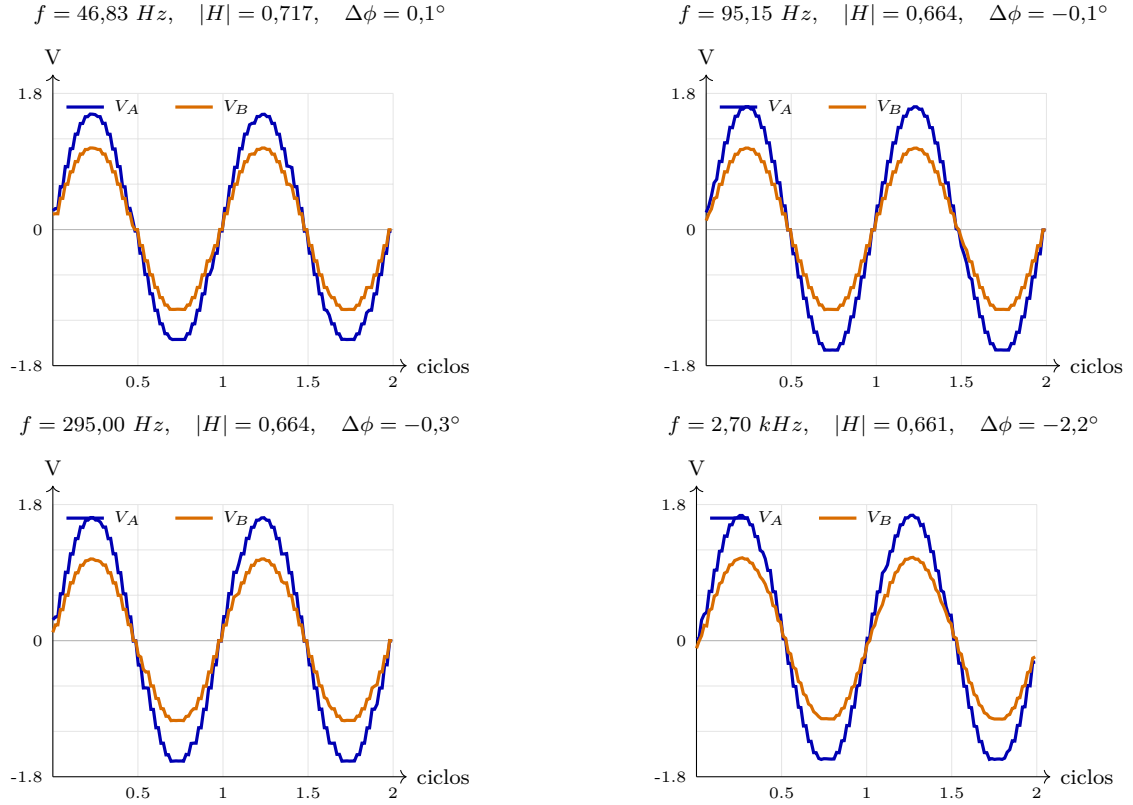


Figura 14: Formas de onda medidas a frecuencias intermedias. La escala común permite comparar directamente la amplitud de entrada y salida.

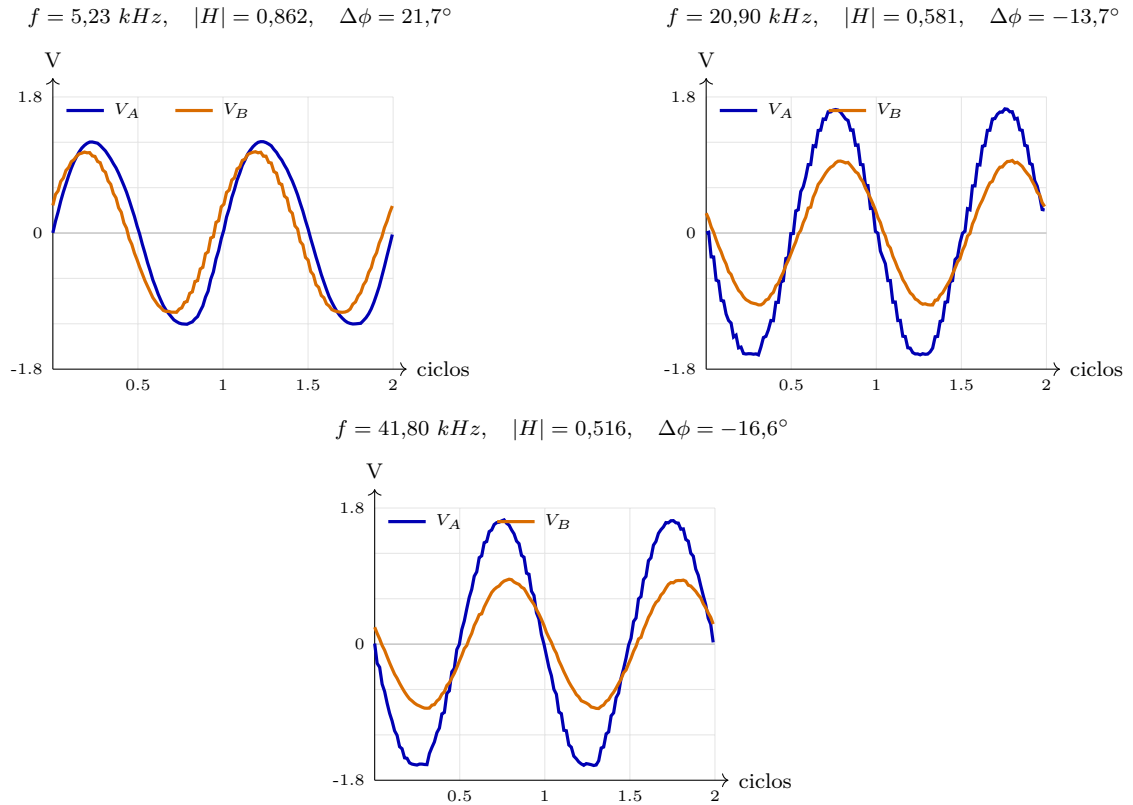


Figura 15: Formas de onda medidas a las frecuencias más elevadas. La captura de $5,23 \text{ kHz}$ presenta un comportamiento anómalo respecto al resto del barrido.

Visualmente se aprecia que las señales permanecen prácticamente en fase en la zona de baja frecuencia. En los dos ensayos de mayor frecuencia la amplitud relativa de V_B disminuye y aparece un retraso de fase más visible. No obstante, el conjunto experimental no presenta una evolución perfectamente monótona: la medida de $5,23\text{ kHz}$ muestra una amplitud y un adelanto de fase anómalos. Por ello, estas formas de onda se presentan como observaciones experimentales y la tendencia se cuantifica posteriormente mediante el análisis fasorial.

Modelo eléctrico y análisis cuantitativo

El análisis teórico se realizó considerando $R_1 = 10\text{ k}\Omega$ como resistencia serie y la asociación $R_{23} = R_2 + R_3 = 20\text{ k}\Omega$ como rama resistiva de la carga. La impedancia equivalente observada desde V_B es:

$$Z_p(f) = \left(\frac{1}{R_{23}} + j2\pi fC \right)^{-1}$$

La relación entre la tensión de entrada y la tensión de salida queda definida por el divisor de impedancias:

$$H(f) = \frac{V_B}{V_A} = \frac{Z_p(f)}{R_1 + Z_p(f)}$$

La frecuencia efectiva de cada captura se estimó a partir de los cruces ascendentes por el valor medio del canal de entrada. Este paso evita emplear directamente la frecuencia redondeada indicada en el nombre del archivo, ya que las frecuencias reales dependen de los divisores enteros de TIM6 y del reloj efectivo del microcontrolador. A continuación, se estimó el fasor de cada canal mediante la componente de Fourier evaluada en la frecuencia medida:

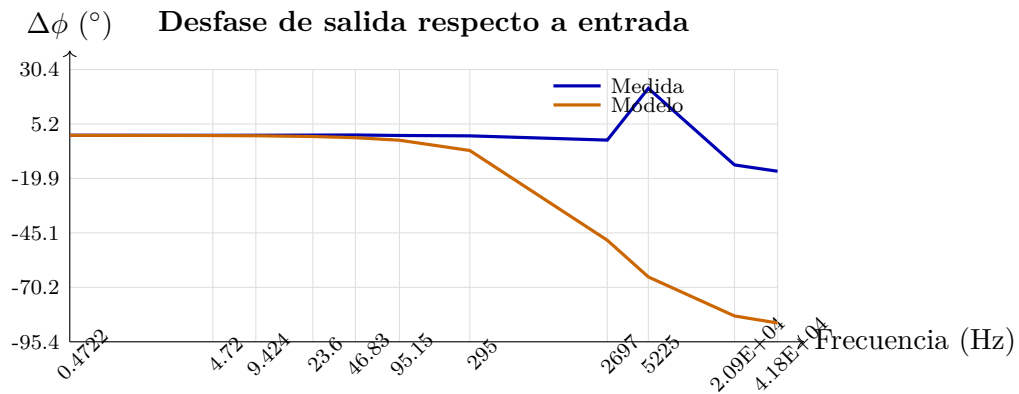
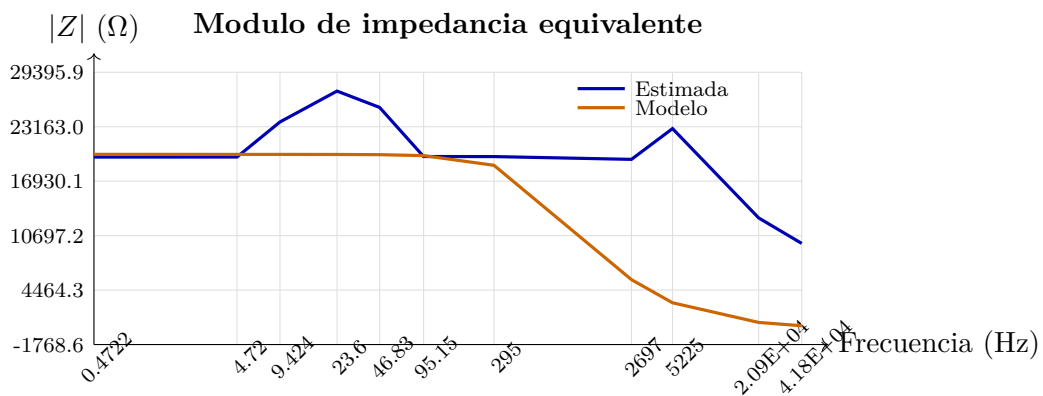
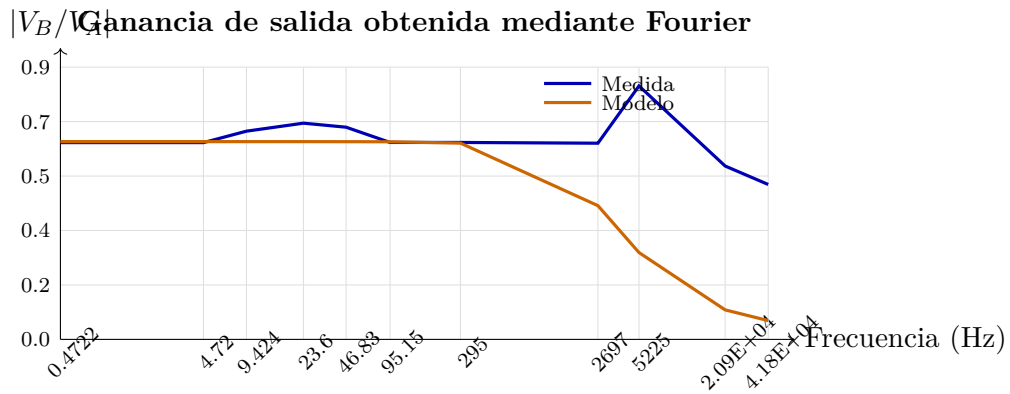
$$X(f_0) = \frac{2}{N} \sum_{n=0}^{N-1} (x[n] - \bar{x}) e^{-j2\pi f_0 t_n}$$

Con los fasores de entrada y salida se calculó $H = V_B/V_A$. Despejando la ecuación anterior se obtuvo una estimación experimental de la impedancia:

$$Z_p(f) = R_1 \frac{H(f)}{1 - H(f)}$$

f_{med} (Hz)	$ H _{med}$	$ H _{teo}$	$ Z _{med}$ (Ω)	$ Z _{teo}$ (Ω)	$\Delta\phi$ ($^\circ$)
0.47	0.663	0.667	19692	20000	0.0
4.72	0.663	0.667	19695	20000	0.0
9.42	0.703	0.667	23713	19999	0.0
23.60	0.732	0.667	27247	19991	0.1
46.83	0.717	0.667	25374	19965	0.1
95.15	0.664	0.666	19754	19859	-0.1
295.00	0.664	0.662	19746	18753	-0.3
2697.08	0.661	0.442	19429	5660	-2.2
5225.39	0.862	0.277	22952	3011	21.7
20896.21	0.581	0.076	12722	761	-13.7
41798.46	0.516	0.038	9807	381	-16.6

Tabla 10: Comparación entre la respuesta medida mediante Fourier y el modelo ideal del circuito.



Los resultados de baja frecuencia concuerdan con la topología real del montaje. Para 0,47 y 4,72 Hz , la ganancia medida es aproximadamente 0,663, frente a 0,667 en el modelo, y la impedancia estimada es de unos 19,7 $k\Omega$, frente a los 20 $k\Omega$ de la rama resistiva. Esta concordancia confirma que la discrepancia obtenida inicialmente procedía de haber situado incorrectamente V_B en el esquema y, por tanto, de haber utilizado una ecuación de divisor que no representaba el circuito medido.

Entre aproximadamente 9 y 47 Hz se observan variaciones de ganancia que no siguen de forma monótona el modelo ideal, aunque a 95 y 295 Hz vuelve a existir una concordancia razonable en magnitud. Por encima de este intervalo aumenta la diferencia entre el modelo con $C = 10$ nF y las medidas. En particular, la captura de 5,23 kHz presenta una ganancia de 0,862 y un adelanto de fase de 21,7°, comportamiento que no es compatible con la red RC pasiva representada y que debe considerarse una medida anómala hasta repetir el ensayo. Las capturas de mayor frecuencia también muestran una atenuación y un desfase muy inferiores a los

previstos para 10 nF , por lo que conviene verificar el valor real del condensador, las conexiones, la referencia común de las sondas y los límites de actualización y adquisición del DAC y del ADC.

4.4. Seguimiento de la planificación

A pesar de la curva de aprendizaje asociada al uso de Rust en sistemas embebidos, el empleo de `defmt` y `probe-rs` permitió agilizar notablemente la depuración del firmware. Frente a una estrategia basada en UART, esta solución evitó dedicar un periférico adicional exclusivamente a la depuración y redujo la cantidad de código necesario para obtener información durante la ejecución. Esto facilitó la validación incremental de los periféricos DAC, ADC, DMA y TIM6, permitiendo detectar errores de configuración de forma más rápida y mantener una implementación más ligera y centrada en el funcionamiento principal del sistema.

4.5. Plan de pruebas

Se realizaron tres pruebas principales:

1. **Prueba de calidad de señal:** Comprobar la estabilidad de frecuencia y observar la distorsión de la señal generada por el DAC.
2. **Prueba de carga:** Medir la respuesta con una resistencia de carga conocida.
3. **Prueba de estrés:** Funcionamiento continuo durante 2 horas para verificar estabilidad del DMA.

5. Cierre del documento

5.1. Conclusiones

Tras la finalización del presente Trabajo de Fin de Grado, se puede concluir que se han cumplido los objetivos principales de desarrollo de un prototipo embebido para generación y adquisición sincronizada de señales sobre la plataforma **STM32G474RE**. La validación con una carga conocida ha permitido comprobar la coherencia del modelo a baja frecuencia y, al mismo tiempo, identificar limitaciones experimentales que deben resolverse antes de considerar el sistema un instrumento de medida de impedancia calibrado.

A continuación, se detallan las conclusiones principales derivadas del desarrollo:

- **Eficacia de la instrumentación integrada:** Las capacidades analógicas del microcontrolador han permitido implementar un demostrador funcional de generación y adquisición con un número reducido de componentes externos. Las medidas de baja frecuencia sobre la carga resistiva-capacitiva concuerdan con el modelo corregido. No obstante, la caracterización cuantitativa a frecuencias elevadas requiere repetir medidas, verificar el valor y conexión de los componentes y estudiar los límites temporales del DAC y del ADC.
- **Robustez del ecosistema Rust:** La implementación del firmware en Rust ha aportado garantías útiles en un entorno sin sistema operativo. La gestión de periféricos de bajo nivel mediante el modelo de *ownership* y el uso de herramientas como `probe-rs` han facilitado la depuración y el control del flujo de ejecución. El uso de `no_std` ha permitido mantener un control directo sobre los recursos del sistema.
- **Rendimiento del hardware:** El uso de **DMA** sincronizado con temporizadores ha permitido reducir la intervención de la CPU durante la generación y la adquisición de muestras. La frecuencia efectiva de cada perfil queda determinada por el reloj de TIM6 y por los

valores enteros de **PSC** y **ARR**; por ello se contrastaron las frecuencias mediante osciloscopio y se corrigieron las etiquetas empleadas por el firmware.

- **Versatilidad del prototipo:** La integración de generación, adquisición y monitorización en una única placa de desarrollo demuestra la viabilidad de continuar el trabajo hacia versiones más compactas, incluyendo una futura PCB dedicada o integraciones en dispositivos de monitorización biomédica.

Como balance final, el proyecto ha permitido construir y validar parcialmente una plataforma experimental de generación y adquisición, además de explorar el uso de Rust en programación embebida de bajo nivel. El prototipo constituye una base útil para continuar hacia un sistema de medida calibrado, incorporando una metodología experimental más completa, acondicionamiento analógico y validación frente a instrumentación de referencia.

5.2. Galería de figuras

Índice de figuras

1.	Diagrama conceptual del sistema	5
2.	Representación fasorial de la impedancia compleja	8
3.	Sincronización entre TIM6, DAC, ADC y DMA. Cada evento de actualización de TIM6 genera el disparo que actualiza el DAC y lanza una conversión del ADC. .	10
4.	Microcontrolador STM32G474RE empleado en el desarrollo del sistema	12
5.	Logotipo del lenguaje Rust.	12
6.	Distribución conceptual de los elementos principales del firmware entre memoria Flash y RAM.	15
7.	Salida por consola durante la ejecución del firmware, mostrando la frecuencia activa y las muestras adquiridas por el ADC.	16
8.	Diagrama de Gantt del proyecto dividido por hitos	22
9.	Arquitectura general del sistema implementado, mostrando la generación mediante DAC, la adquisición mediante ADC y la sincronización común mediante TIM6. .	23
10.	Flujo de selección cíclica de perfiles de frecuencia mediante el pulsador de usuario conectado a PC13.	25
11.	Montaje experimental utilizado durante las pruebas con la Analog Discovery 3 de Digilent.	26
12.	Circuito de prueba utilizado para las medidas de impedancia.	27
13.	Formas de onda medidas a baja frecuencia. Se representan las componentes alternas de V_A y V_B durante dos periodos, manteniendo la misma escala vertical. .	27
14.	Formas de onda medidas a frecuencias intermedias. La escala común permite comparar directamente la amplitud de entrada y salida.	28
15.	Formas de onda medidas a las frecuencias más elevadas. La captura de $5,23\text{ kHz}$ presenta un comportamiento anómalo respecto al resto del barrido.	28
16.	Ferris, mascota no oficial asociada al ecosistema Rust.	34

Referencias

- [1] W. H. Hayt, J. E. Kemmerly, and S. M. Durbin, *Análisis de circuitos en ingeniería*. McGraw-Hill, 8 ed., 2012.
- [2] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*. Pearson, 3 ed., 2010.
- [3] STMicroelectronics, *RM0440 Reference manual: STM32G4 series advanced Arm-based 32-bit MCUs*, 2023. Rev 7.
- [4] S. Klabnik and C. Nichols, *The Rust Programming Language*. No Starch Press, 2019.
- [5] M. Matthes, *Hands-On Rust for Embedded Systems*. Packt Publishing, 2023.
- [6] S. Grimnes and O. G. Martinsen, “Bioimpedance and bioelectricity basics,” *Academic Press*, 2014.

A. Anexos

A.1. Implementación del Firmware en Rust



Figura 16: Ferris, mascota no oficial asociada al ecosistema Rust.

En estos anexos se detalla el código fuente principal desarrollado para el microcontrolador **STM32G474RE**, así como otros detalles técnicos. La implementación destaca por el uso de abstracciones de coste cero y por la seguridad de memoria proporcionada por el compilador de Rust [4].

A.2. Configuración de Periféricos y Bucle Principal

El firmware principal desarrollado para el microcontrolador **STM32G474RE** se encarga de inicializar los periféricos necesarios para la generación y adquisición de señales analógicas. La implementación se realiza en Rust en un entorno *bare-metal*, sin sistema operativo ni biblioteca estándar, accediendo directamente a los registros del microcontrolador mediante la capa PAC proporcionada por la familia **stm32g4**.

El objetivo de esta configuración es coordinar varios periféricos internos del microcontrolador para generar una señal senoidal mediante el **DAC**, adquirir la señal resultante mediante el **ADC** y automatizar ambas operaciones mediante **DMA**. Para garantizar una temporización estable, se utiliza el temporizador **TIM6** como fuente de eventos de disparo.

Inicialización del sistema

Al comienzo del programa se obtiene el acceso a los periféricos del microcontrolador mediante:

Listing 1: Código principal main.rs para adquisición biomédica.

```
#![no_std]
#![no_main]

use cortex_m::singleton;
use cortex_m_rt::entry;
use defmt_rtt as _;
use panic_probe as _;
use stm32g4::stm32g474::{self, gpioc};

struct PerfilFrecuencia {
    frecuencia_hz: u32,
    psc: u16,
    arr: u32,
}

type GpioC = stm32g4::Periph<gpioc::RegisterBlock, 1207961600>;
```

```

const N_MUESTRAS: usize = 32;
const PERFIL_INICIAL: usize = 2;

const TIM6_PSC_INICIAL: u16 = 169;
const TIM6_ARR_INICIAL: u32 = 100;
const TIM6_TRGO_UPDATE: u8 = 0b010;

const DAC_VALOR_MEDIO: u32 = 2048;

const DMA_CH_ADC: usize = 0;
const DMA_CH_DAC: usize = 2;
const DMA_MEMORIA_16_BITS: u8 = 0b01;
const DMA_PERIFERICO_32_BITS: u8 = 0b10;
const DMA_PRIORIDAD_ALTA: u8 = 0b10;
const DMAMUX_REQ_ADC12: u8 = 5;
const DMAMUX_REQ_DAC1_CH1: u8 = 6;

const ADC_CANAL_PA0: u8 = 1;
const ADC_CKMODE_PCLK_DIV2: u8 = 0b01;
const ADC_SAMPLING_MODERADO: u8 = 0b010;
const DELAY_ADC_REGULADOR: u32 = 30_000;
const DELAY_ADC_CALIBRACION: u32 = 20_000;

const DELAY_BUCLE_PRINCIPAL: u32 = 1_000_000;

const TABLA_SENO: [u16; N_MUESTRAS] = [
    2048, 2447, 2831, 3185, 3495, 3750, 3939, 4056, 4095, 4056, 3939, 3750, 3495,
    3185, 2831, 2447, 2048, 1649, 1265, 911, 601, 346, 157, 40, 0, 40, 157, 346,
    601, 911, 1265, 1649,
];

// Frecuencias redondeadas a partir de las lecturas del osciloscopio.
const PERFILES: [PerfilFrecuencia; 10] = [
    PerfilFrecuencia { frecuencia_hz: 5,      psc: 169, arr: 624, },
    PerfilFrecuencia { frecuencia_hz: 9,      psc: 169, arr: 312, },
    PerfilFrecuencia { frecuencia_hz: 24,     psc: 169, arr: 124, },
    PerfilFrecuencia { frecuencia_hz: 47,     psc: 169, arr: 62, },
    PerfilFrecuencia { frecuencia_hz: 95,     psc: 169, arr: 30, },
    PerfilFrecuencia { frecuencia_hz: 295,    psc: 169, arr: 9, },
    PerfilFrecuencia { frecuencia_hz: 2697,   psc: 5,  arr: 30, },
    PerfilFrecuencia { frecuencia_hz: 5225,   psc: 5,  arr: 15, },
    PerfilFrecuencia { frecuencia_hz: 20896,  psc: 5,  arr: 3, },
    PerfilFrecuencia { frecuencia_hz: 41798,  psc: 5,  arr: 1, },
];

fn habilitar_relojes(rcc: &stm32g474::RCC) {
    rcc.ahb1enr().modify(|_, w| {
        w.dma1en().set_bit();
        w.dmamux1en().set_bit();
        w
    })
}

```

```

});

rcc.ahb2enr().modify(|_, w| {
    w.dac1en().set_bit();
    w.gpioaen().set_bit();
    w.adc12en().set_bit();
    w.gpiocen().set_bit();
    w
});

rcc.apb1enr1().modify(|_, w| {
    w.tim6en().set_bit();
    w
});
}

fn configurar_gpioa(gpioa: &stm32g474::GPIOA) {
    gpioa.moder().modify(|_, w| {
        w.moder4().analog(); // DAC1_OUT1 (PA4)
        w.moder0().analog(); // ADC1_IN1 (PA0)
        w
    });
}

fn configurar_boton(gpioc: &GpioC) {
    gpioc.moder().modify(|_, w| {
        w.moder13().input();
        w
    });
}

fn configurar_tim6(tim6: &stm32g474::TIM6) {
    unsafe {
        tim6.psc().write(|w| w.psc().bits(TIM6_PSC_INICIAL));
        tim6.arr().write(|w| w.arr().bits(TIM6_ARR_INICIAL));
        tim6
            .cr2()
            .modify(|_, w| w.mms().bits(TIM6_TRGO_UPDATE));
        tim6.dier().modify(|_, w| w.ude().set_bit());
        tim6.cr1().modify(|_, w| w.cen().set_bit());
    }
}

fn aplicar_perfil_frecuencia(tim6: &stm32g474::TIM6, perfil: &PerfilFrecuencia) {
    unsafe {
        tim6.cr1().modify(|_, w| w.cen().clear_bit());
        tim6.psc().write(|w| w.psc().bits(perfil.psc));
        tim6.arr().write(|w| w.arr().bits(perfil.arr));
        tim6.cnt().write(|w| w.cnt().bits(0));
        tim6.egr().write(|w| w.ug().set_bit());
        tim6
    }
}

```

```

        .cr2()
        .modify(|_, w| w.mms().bits(TIM6_TRGO_UPDATE));
    tim6.cr1().modify(|_, w| w.cen().set_bit());
}

}

fn configurar_dac(dac: &stm32g474::DAC1) {
    unsafe {
        dac.dhr12r1().write(|w| w.bits(DAC_VALOR_MEDIO));
        dac.cr().modify(|_, w| {
            w.en1().set_bit();
            w.ten1().set_bit();
            w.dmaen1().set_bit();
            w.tsel1().tim6trgo();
            w
        });
    }
}

fn configurar_dma_dac(dma1: &stm32g474::DMA1, dac: &stm32g474::DAC1) {
    let dac_dhr12_addr = dac.dhr12r1().as_ptr() as u32;

    unsafe {
        dma1.ch(DMA_CH_DAC).cr().modify(|_, w| w.en().clear_bit());
        dma1
            .ch(DMA_CH_DAC)
            .par()
            .write(|w| w.pa().bits(dac_dhr12_addr));
        dma1
            .ch(DMA_CH_DAC)
            .mar()
            .write(|w| w.ma().bits(TABLA_SENO.as_ptr() as u32));
        dma1
            .ch(DMA_CH_DAC)
            .ndtr()
            .write(|w| w.ndt().bits(N_MUESTRAS as u16));

        dma1.ch(DMA_CH_DAC).cr().modify(|_, w| {
            w.minc().set_bit();
            w.circ().set_bit();
            w.dir().set_bit();
            w.msize().bits(DMA_MEMORIA_16_BITS);
            w.psize().bits(DMA_PERIFERICO_32_BITS);
            w.pl().bits(DMA_PRIORIDAD_ALTA);
            w.en().set_bit();
            w
        });
    }
}

fn configurar_dmamux(dmamux: &stm32g474::DMAMUX) {

```

```

unsafe {
    dmamux.ccr(DMA_CH_ADC).modify(|_, w| {
        w.dmareq_id().bits(DMAMUX_REQ_ADC12);
    });

    dmamux.ccr(DMA_CH_DAC).modify(|_, w| {
        w.dmareq_id().bits(DMAMUX_REQ_DAC1_CH1);
    });
}

fn configurar_adc(adc: &stm32g474::ADC1, adc_common: &stm32g474::ADC12_COMMON) {
    unsafe {
        adc_common
            .ccr()
            .modify(|_, w| w.ckmode().bits(ADC_CKMODE_PCLK_DIV2));
    }

    if adc.cr().read().aden().bit_is_set() {
        adc.cr().modify(|_, w| w.addis().set_bit());
        while adc.cr().read().aden().bit_is_set() {}
    }

    adc.cr().modify(|_, w| w.deeppwd().clear_bit());
    adc.cr().modify(|_, w| w.advregen().set_bit());
    cortex_m::asm::delay(DELAY_ADC_REGULADOR);

    adc.cr().modify(|_, w| w.adcal().set_bit());
    while adc.cr().read().adcal().bit_is_set() {}
    cortex_m::asm::delay(DELAY_ADC_CALIBRACION);

    adc.sqr1().modify(|_, w| unsafe {
        w.l().bits(0);
        w.sq1().bits(ADC_CANAL_PA0)
    });

    unsafe {
        adc.smpr1().modify(|_, w| {
            w.smp1().bits(ADC_SAMPLING_MODERADO);
        });
    }

    adc.cfgr().modify(|_, w| {
        w.cont().clear_bit();
        w.dmaen().set_bit();
        w.dmacfg().set_bit();
        w.extsel().tim6_trgo();
        w.exten().rising_edge();
    });
}

```

```

        w
    });

    adc.isr().write(|w| w.adrdy().clear());
    adc.cr().modify(|_, w| w.aden().set_bit());
}

fn configurar_dma_adc(
    dma1: &stm32g474::DMA1,
    adc: &stm32g474::ADC1,
    adc_buffer: &[u16; N_MUESTRAS],
) {
    let adc_dr_addr = adc.dr().as_ptr() as u32;

    unsafe {
        dma1.ch(DMA_CH_ADC).cr().modify(|_, w| w.en().clear_bit());
        dma1
            .ch(DMA_CH_ADC)
            .par()
            .write(|w| w.pa().bits(adc_dr_addr));
        dma1
            .ch(DMA_CH_ADC)
            .mar()
            .write(|w| w.ma().bits(adc_buffer.as_ptr() as u32));
        dma1
            .ch(DMA_CH_ADC)
            .ndtr()
            .write(|w| w.ndt().bits(N_MUESTRAS as u16));

        dma1.ch(DMA_CH_ADC).cr().modify(|_, w| {
            w.minc().set_bit();
            w.circ().set_bit();
            w.dir().clear_bit();
            w.msize().bits(DMA_MEMORIA_16_BITS);
            w.psize().bits(DMA_PERIFERICO_32_BITS);
            w.pl().bits(DMA_PRIORIDAD_ALTA);
            w.en().set_bit();
            w
        });
    }
}

fn arrancar_adc(adc: &stm32g474::ADC1) {
    adc.cr().modify(|_, w| w.adstart().set_bit());
}

fn leer_buffer_adc(adc_buffer: &[u16; N_MUESTRAS]) -> &[u16] {
    unsafe { core::slice::from_raw_parts(adc_buffer.as_ptr(), N_MUESTRAS) }
}

#[entry]

```

```

fn main() -> ! {
    defmt::println!("Iniciando STM32G474...");
    let dp = stm32g474::Peripherals::take().unwrap();

    // Se aplica el perfil inicial antes de arrancar la generación sincronizada.
    let mut perfil_actual = PERFIL_INICIAL;

    let rcc = dp.RCC;
    habilitar_relojes(&rcc);

    let gpioa = dp.GPIOA;
    configurar_gpioa(&gpioa);

    let tim6 = dp.TIM6;
    configurar_tim6(&tim6);
    aplicar_perfil_frecuencia(&tim6, &PERFILES[perfil_actual]);

    let dac = dp.DAC1;
    configurar_dac(&dac);

    let dma1 = dp.DMA1;
    configurar_dma_dac(&dma1, &dac);

    let dmamux = dp.DMAMUX;
    configurar_dmamux(&dmamux);

    let adc = dp.ADC1;
    let adc_common = dp.ADC12_COMMON;
    configurar_adc(&adc, &adc_common);

    let adc_buffer = singleton!(u16; N_MUESTRAS] = [0; N_MUESTRAS]).unwrap();
    configurar_dma_adc(&dma1, &adc, adc_buffer);

    let gpioc: GpioC = dp.GPIOC;
    configurar_boton(&gpioc);

    arrancar_adc(&adc);
    defmt::println!("ADC en marcha");

    loop {
        cortex_m::asm::delay(DELAY_BUCLE_PRINCIPAL);

        let boton_pulsado = gpioc.idr().read().idr13().bit_is_set();

        if boton_pulsado {
            perfil_actual = (perfil_actual + 1) % PERFILES.len();
            aplicar_perfil_frecuencia(&tim6, &PERFILES[perfil_actual]);
        }

        let lectura_adc = leer_buffer_adc(adc_buffer);
        defmt::println!(

```



```

        "Frecuencia_{{?:?}}Hz:{{?:?}}",
        PERFILES[perfil_actual].frecuencia_hz,
        lectura_adc
    );
}
}

```

A.3. Gestión de perfiles de frecuencia y pulsador de usuario

El listado anterior incluye la gestión completa de perfiles de frecuencia. Cada perfil almacena la frecuencia nominal y los valores PSC/ARR aplicados a TIM6. La tabla de perfiles se declara como constante, por lo que queda ubicada junto al resto de datos inmutables del firmware y puede consultarse sin asignación dinámica.

El pulsador de usuario conectado a PC13 permite avanzar de forma circular por la tabla de perfiles. Al seleccionar un nuevo perfil, el firmware detiene temporalmente TIM6, actualiza sus registros de temporización, reinicia el contador y genera un evento de actualización. Como DAC y ADC dependen del mismo disparo hardware, el cambio de frecuencia mantiene la sincronización entre generación y adquisición.

A.4. Detalles de la Síntesis Digital Directa (DDS)

La señal senoidal se genera mediante una transferencia circular por DMA[3]. La tabla contiene 32 muestras por periodo, por lo que la frecuencia de salida depende del reloj de TIM6 y de los registros de prescaler y auto-recarga:

$$f_{out} = \frac{f_{TIM6}}{(PSC + 1)(ARR + 1)N_{muestras}}$$

La ventaja de este enfoque es que la forma de onda permanece definida por una única tabla, mientras que la frecuencia se modifica actuando únicamente sobre TIM6. Así se evita recalcular muestras durante la ejecución y se mantiene una carga reducida sobre la CPU.

A.5. Uso de inteligencia artificial generativa

Durante el desarrollo de este Trabajo de Fin de Grado se ha utilizado inteligencia artificial generativa como herramienta de apoyo, principalmente en tareas auxiliares relacionadas con la organización, revisión y mejora del material elaborado. Su uso se ha limitado a aspectos formales y de apoyo, sin delegar en ella el diseño técnico ni la implementación funcional del firmware.

En concreto, la inteligencia artificial generativa se ha empleado para mejorar la redacción de determinados apartados de la memoria, ordenar explicaciones técnicas, revisar la coherencia del texto y proponer formas más claras de estructurar algunos contenidos. También se ha utilizado como apoyo para ordenar fragmentos de código, mejorar la legibilidad de ciertos bloques y renombrar variables o funciones con nombres más descriptivos y coherentes con la finalidad de cada elemento.

Sin embargo, no se ha utilizado inteligencia artificial generativa para crear el código funcional del proyecto. Esta decisión se debe a las características específicas del trabajo desarrollado. Aunque existe una gran cantidad de información disponible sobre programación embebida en microcontroladores STM32, la documentación y los ejemplos específicos que combinan STM32 con Rust embebido son mucho más limitados. Como consecuencia, las herramientas de IA generativa tienden a producir respuestas aparentemente correctas, pero con errores técnicos importantes, uso incorrecto de registros, llamadas inexistentes a APIs, configuraciones incompatibles con el microcontrolador o soluciones que no compilan en un entorno `no_std`.

Durante las pruebas realizadas con este tipo de herramientas se observó que, en el contexto concreto de Rust embebido sobre STM32, era frecuente la aparición de respuestas erróneas o alucinadas. Por ejemplo, la IA podía mezclar conceptos procedentes de bibliotecas de alto nivel, ejemplos en C basados en HAL, APIs de otros microcontroladores o configuraciones que no correspondían al dispositivo empleado. Este comportamiento hacía que el código generado no fuese fiable como base de implementación.

Por este motivo, la implementación del firmware se realizó manualmente, consultando la documentación técnica del microcontrolador, los registros de los periféricos y el comportamiento esperado del sistema. La IA generativa se mantuvo como una herramienta de apoyo para mejorar la presentación, la claridad y la organización del trabajo, pero no como fuente principal para el desarrollo del código ni para la toma de decisiones técnicas críticas.

De esta forma, el uso de inteligencia artificial generativa en el proyecto puede considerarse equivalente a una herramienta de asistencia en la edición y revisión del documento, pero no como un sustituto del proceso de diseño, programación, validación y depuración realizado durante el desarrollo del sistema.